

Dati e Algoritmi

[Appunti di Piai Luca]

Academic Year 2022-2023

Indice

1	Introduzione	5
1.1	Problemi computazionali	5
1.2	Algoritmo e modello di calcolo	5
1.2.1	Correttezza di un algoritmo	5
1.2.2	Prestazioni di un algoritmo	6
1.2.3	Dimensione di un problema	6
1.2.4	Modello computazionale RAM	6
1.3	Pseudocodice	6
2	Analisi di algoritmi	8
2.1	Analisi teorica delle prestazioni	8
2.1.1	Andamento asintotico delle prestazioni	8
2.1.2	Analisi di algoritmi ricorsivi	9
2.1.3	Cenni sugli algoritmi deterministici	9
2.1.4	Prestazioni temporali asintotiche di alcuni algoritmi operanti su array	9
3	Liste	11
3.1	Introduzione	11
3.2	Lista con indice o con rango	11
3.3	Lista con posizione astratta	12
3.4	Iteratore di una lista	13
3.5	IndexList o PositionalList?	14
4	Alberi	15
4.1	Introduzione	15
4.2	L'interfaccia Tree	16
4.3	Profondità di un nodo	18
4.4	Altezza di un nodo	18
4.5	Attraversamenti di alberi	19
4.5.1	Attraversamento in pre-ordine	19
4.5.2	Attraversamento in post-ordine	21
4.5.3	Attraversamento per livelli o in ampiezza	22
4.6	Alberi binari	23
4.6.1	Attraversamento in ordine simmetrico	24
4.6.2	L'interfaccia BinaryTree. Due possibili implementazioni	24
5	Code prioritarie	27
5.1	Relazioni d'ordine	27
5.1.1	Definire relazioni d'ordine in Java	27
5.2	Introduzione alle code prioritarie	28
5.3	L'interfaccia Entry	28

5.4	L'interfaccia PriorityQueue	29
5.5	Heap	30
5.5.1	Definizione di Heap	30
5.5.2	L'interfaccia CompleteBinaryTree	30
5.6	Realizzazione tramite Heap	31
5.7	Priority Queue Sort	32
6	Mappe	33
6.1	Introduzione	33
6.2	L'interfaccia Map	33
6.2.1	L'interfaccia SortedMap	34
6.3	Alberi binari di ricerca	34
6.3.1	SortedMap con BST	35
6.3.2	Alberi AVL	36
7	Ordinamenti	40
7.1	Bucket Sort	40
7.2	Paradigma divide and conquer	41
7.3	Merge-Sort	41
7.4	Quick-Sort	43
8	Hasing	46
8.1	Tabelle	46
8.2	Funzioni hash	46
8.2.1	Funzione di codifica	46
8.2.2	Funzioni di compressione	47
8.2.3	Gestione collisioni	47
9	Grafi	49
9.1	Introdizione	49
9.2	L'interfaccia Graph	50
9.3	Attraversamenti di grafi	51
9.3.1	Attraversamento DFS	51
9.3.2	Attraversamento BFS	52
9.4	Implementazione di grafi	53
9.5	L'algoritmo di Dijkstra	54

Capitolo 1

Introduzione

1.1 Problemi computazionali

I calcolatori nascono per risolvere problemi computazionali, cioè problemi di calcolo.

Un problema computazionale P è definito come una relazione tra un insieme I di esemplari (o istanze) del problema stesso e un insieme S di soluzioni. $P \subseteq I \times S$, con il vincolo che $\forall i \in I \exists s \in S \mid (i, s) \in P$.

Per esempio possiamo formalizzare il calcolo della somma di due numeri interi: $I = \mathbb{Z} \times \mathbb{Z}$ rappresenta l'insieme di tutte le coppie possibili di numeri interi; $S = \mathbb{Z}$ rappresenta l'insieme di tutti i risultati possibili (cioè l'insieme dei numeri interi). Quindi possiamo esprimere P come $P = \{((a, b), c) \mid a, b, c \in \mathbb{Z} \wedge c = a + b\} \subset I \times S = (\mathbb{Z} \times \mathbb{Z}) \times \mathbb{Z}$.

In questo caso P è sottoinsieme proprio perché esiste un unico numero naturale che è la somma di altri due. Se avessimo avuto che $P = I \times S$ allora significava che qualsiasi esemplare i potevo associare una qualsiasi soluzione s , in questo specifico esempio non ha senso.

L'informatica teorica ci dice che se un problema è risolubile al calcolatore, allora è descrivibile sottoforma di problema computazionale. In generale non è sempre facile descrivere i problemi sottoforma di problema computazionale, inoltre, in alcuni casi, non agevola l'intuizione per la creazione di un algoritmo.

1.2 Algoritmo e modello di calcolo

Un algoritmo A che risolve un problema computazionale $P \subseteq I \times S$ è una procedura che, dato un esemplare $i \in I$, individua tramite una sequenza finita e non ambigua di passi computazionali elementari una soluzione $s \in S$ tale che $(i, s) \in P$.

Si dice anche che un algoritmo risolve un problema $P \subseteq I \times S$ se e solo se calcola una funzione f_A tale che $\forall i \in I, (i, f_A(i)) \in P$. In generale non ci interessa cosa calcola f_A quando $i \notin I$, in quel caso si parla di violazione delle pre-condizioni del problema. Non sempre un algoritmo verifica che $i \in I$, in alcuni casi potrebbe essere causa di un peggioramento delle prestazioni.

1.2.1 Correttezza di un algoritmo

Un algoritmo corretto risolve il problema in ogni caso possibile che si possa presentare, equivalentemente si dice che ha copertura 100% (cioè è verificato $\forall i \in I$) e che il collaudo fatto è esaustivo. Un metodo per verificare che un algoritmo è corretto è quello di dimostrarlo formalmente, ma spesso è molto difficile.

Per dimostrare la correttezza di un algoritmo usiamo delle tecniche, per esempio: esempio o controesempio; deduzione per assurdo; ragionamento per induzione; condizione invariante; ecc.

1.2.2 Prestazioni di un algoritmo

Esistono diversi algoritmi che risolvono uno stesso problema, questi si possono differenziare in base alle loro prestazioni. In generale le prestazioni si possono valutare in termini di tempo di esecuzione, occupazione della memoria, consumo di energia e altro ancora. Il nostro target principale è il tempo di esecuzione.

In generale le prestazioni di un algoritmo dipendono dalla dimensione o taglia dell'esemplare che viene risolto.

1.2.3 Dimensione di un problema

Per definizione la dimensione di un problema computazionale è una funzione che assegna a ciascun esemplare del problema uno o più numeri (solitamente interi) che forniscono una misura ragionevole dell'esemplare stesso.

La dimensione è una misura ragionevole degli esemplari di un problema se le prestazioni dell'algoritmo che risolvono il problema dipendono soltanto dalla dimensione stessa. (Significa che per esempio se ho un algoritmo che opera su un array, la dimensione del problema è la lunghezza dell'array stesso e le prestazioni dell'algoritmo dipendono solo dalla dimensione e non dai dati presenti).

Per le strutture dati lineari definiamo dimensione il numero di dati contenuti nella struttura, per strutture bidimensionali (es: matrice) la dimensione può essere data da due numeri interi (righe e colonne) oppure da un unico numero intero (area).

1.2.4 Modello computazionale RAM

Un modello computazionale è un'astrazione di un dispositivo di calcolo caratterizzato da un insieme di istruzioni o operazioni eseguibili. Il modello che utilizzeremo è il calcolatore *RAM* (*Random Access Machine*), questa descrive la macchina sulla quale valuteremo gli algoritmi. Le istruzioni elementari che comprende sono:

- Operazioni logiche e aritmetiche elementari eseguite in un tempo $\Theta(1)$ indipendentemente dai valori elaborati;
- Occupazione di memoria $\Theta(1)$ per qualsiasi valore numerico;
- Trasferimento di dati tra CPU e memoria nelle due possibili direzioni (lettura e scrittura in memoria);
- Tempo di accesso agli elementi di un array indipendente dall'indice;
- Esecuzione condizionale per effetto di confronti tra dati (anche iterazioni);
- Possibilità di ricorsione (salti e memorizzazione di un punto di ritorno);

1.3 Pseudocodice

Chiamiamo pseudocodice l'insieme di istruzioni del modello RAM. Questo linguaggio, a differenza di quelli tradizionali, è destinato alla lettura umana e non al compilatore. Infatti il suo scopo è quello di analizzare un algoritmo e non di verificarne la correttezza.

Lo pseudocodice è caratterizzato da una sintassi non rigida, siamo liberi di scegliere le convenzioni che più ci piacciono purché comprensibili al programmatore che lo leggerà. Benché non ci sia rigido, dobbiamo comunque seguire alcune direttive:

- le parentesi grafe per identificare i blocchi non sono necessarie, bisogna indentare correttamente il codice;
- specifico di tipi delle variabili solo quando è rilevante;
- evitare variabili globali se non strettamente necessarie (nel nostro corso sono proibite se non specificato nel problema), è dimostrato che introducono errori;
- scegliere un simbolo di assegnazione ($=$, $:=$, $\leftarrow$, $\leftarrow$);
- scegliere un simbolo di uguaglianza matematica ($==$, $=$);
- come operatore indice negli array usare le parentesi quadre, per la lunghezza si può scegliere (`a.length`, `size(a)`, `length(a)`, `|a|`);
- per i cili si usa la sintassi di java/c/c++, le parentesi tonde possono essere omettere, si può usare anche il `for each`;
- per i metodi usiamo le tonde, i parametri li separiamo con una virgola e bisogna specificare sempre il tipo. Usiamo `return` per restituire valori;
- per creare gli oggetti usiamo l'operatore `new`, per accedere ai campi o ai metodi usiamo il punto;
- commentiamo con `//`.

Capitolo 2

Analisi di algoritmi

2.1 Analisi teorica delle prestazioni

Il tempo di esecuzione di un algoritmo dipende dal tipo di istruzioni in linguaggio macchina eseguite dal processore, questo tempo varia a seconda dalla macchina su cui viene eseguito l'algoritmo. Per questo motivo, per valutare le prestazioni degli algoritmi, siamo costretti a fare delle semplificazioni drastiche.

Durante l'esecuzione del codice dell'algoritmo, contiamo soltanto gli accessi in lettura o scrittura a singoli elementi della struttura dati. Nel caso specifico degli array, le istruzioni costose sono quelle di accesso in lettura e scrittura (poniamo che abbiamo lo stesso tempo) ai singoli elementi, il resto è trascurabile.

Se l'algoritmo non opera su array bisogna individuare quali sono le operazioni costose della struttura che stiamo considerando (es: per le liste è l'accesso ai nodi).

2.1.1 Andamento asintotico delle prestazioni

Nell'analisi degli algoritmi ci concentriamo sulla velocità di crescita del tempo d'esecuzione in funzione della dimensione n del nostro problema.

Per confrontare le prestazioni temporali utilizziamo delle notazioni che ci permettono di stimare l'andamento qualitativo al tendere di n all'infinito.

- O-grande: $f(n) \in O(g(n))$ se $\exists c > 0$ e $k > 0$ tali che $f(n) \leq cg(n) \forall n \geq k$. Ci permette di affermare che $f(n)$ è minore di o uguale a $g(n)$ per n che tende all'infinito.
- Omega: $f(n) \in \Omega(g(n))$ se $\exists c > 0$ e $k > 0$ tali che $f(n) \geq cg(n) \forall n \geq k$. Ci permette di affermare che $f(n)$ è maggiore di o uguale a $g(n)$ per n che tende all'infinito.
- Theta: $f(n) \in \Theta(g(n))$ se $f(n) \in \Omega(g(n))$ e $f(n) \in O(g(n))$. Consente di dire che due funzioni crescono con la stessa velocità.

Nel caso di funzioni di più variabili, le notazioni asintotiche si estendono in modo naturale.

Le funzioni $f(n)$ che andremo a considerare saranno polinomi mentre $g(n)$ sarà sempre un monomio, un monomio moltiplicato per il logaritmo oppure esponenziale, questo per confrontare algoritmi sulla base dell'andamento delle loro prestazioni. Con questi vincoli si ottengono funzioni che non appartengono ad alcun insieme Θ , anche se per definizione matematica si ha che ogni funzione è Θ di se stessa.

Algoritmi ottimi:

Parliamo di algoritmi ottimi a livello di prestazioni quando "non si può fare meglio di così" asintoticamente. Per dimostrare che un algoritmo è asintoticamente ottimo bisogna mostrare

che il suo limite superiore coincide con quello inferiore, in altre parole bisogna trovare $\Theta(g(n))$ per una qualche funzione $g(n)$.

Nota che trovare un limite inferiore è molto importante in quanto ci permette di dire che per quel dato problema è inutile cercare un algoritmo che abbia prestazioni migliori.

2.1.2 Analisi di algoritmi ricorsivi

Nel caso di algoritmi ricorsivi, per stimare le prestazioni, si contano le singole operazioni che vengono eseguite durante una specifica attivazione, cioè durante una singola invocazione ricorsiva.

Prendiamo un esempio: $T(n) = T(\frac{n}{2}) + 1$, una funzione scritta in forma implicita. Riuscire a scrivere $T(n)$ in funzione di n è un problema che riguarda la matematica discerta, per questo motivo, in questi casi, ci affideremo alla matematica e ci limiteremo a verificare per sostituzione la correttezza delle forme esplicite fornite.

Per quanto riguarda lo spazio, gli algoritmi ricorsivi necessitano di spazio "implicito" (runtime stack). Indipendentemente dal fatto che nel metodo vengano dichiarate variabili (l'indirizzo di ritorno deve comunque essere memorizzato ad ogni chiamata), ogni chiamata richiede una quantità di spazio fissa. Per trovare la memoria totale occupata dall'algoritmo dobbiamo: calcolare la memoria occupata in una invocazione; conoscere la profondità di ricorsione massima (es: ricerca per bisezione ricorsiva $\rightarrow \log_2 n$, cioè richiede uno spazio aggiuntivo $O(\log n)$).

2.1.3 Cenni sugli algoritmi deterministici

Un algoritmo deterministico è un algoritmo che, per uno stesso esemplare di un problema, trova sempre la stessa soluzione tra tutte quelle possibili. Gli algoritmi non deterministici possono calcolare soluzioni diverse di volta in volta, questo approccio viene sfruttato al fine di migliorare le prestazioni.

Per capire il concetto analizziamo un semplice esempio: poniamo di voler trovare la posizione di uno tra gli elementi massimi all'interno di un array. Un algoritmo deterministico potrebbe (per esempio) fornire la posizione del primo elemento più grande che trova, la cosa importante da sottolineare è che esso troverà sempre la stessa posizione, ovviamente. Non vale la stessa cosa se l'algoritmo è non deterministico, in questo caso potrebbe fornire una delle posizioni in cui si trova il massimo, non è detto che fornisca sempre la stessa.

2.1.4 Prestazioni temporali asintotiche di alcuni algoritmi operanti su array

Riassumiamo alcune complessità di algoritmi comuni che utilizzeremo in combinazione con altri più complessi.

- Clonazione di un array: in generale dipende dal linguaggio. In questo corso ci basiamo su Java: quando dichiariamo un array, questo viene inizializzato automaticamente con i valori di default, successivamente viene sovrascritto con i valori dell'array da clonare. Vengono effettuate le seguenti operazioni: lettura array da clonare (n), inizializzazione nuovo array (n) e clonazione array (n) $\Rightarrow \Theta(n)$.
- Ridimensionamento: nella maggior parte dei linguaggi bisogna creare un nuovo array più grande. Solitamente si raddoppia la dimensione. Le operazioni che vengono effettuate sono: creazione e inizializzazione di un nuovo array lungo $2n$, lettura e scrittura di n valori dell'array da clonare $\Rightarrow 2n + n + n \Rightarrow \Theta(n)$.
- Ricerca di minimo o massimo: è necessario leggere una sola volta tutti gli elementi di un array $\Rightarrow \Theta(n)$.

- Inserimento di un elemento: nel caso migliore (c'è spazio libero) $\Theta(1)$ e nel caso peggiore (se devo ridimensionare) $\Theta(n)$. Per il caso medio bisogna ricorrere all'analisi ammortizzata, in particolare si dimostra che se ridimensiono l'array per un fattore moltiplicativo (solitamente 2 in quanto rappresenta un buon compromesso tra spazio investito e tempo richiesto) si ottiene una funzione $\Theta(1)$ in media.
- Rimozione e inserimento di elementi da un array: bisogna coordinare il fattore moltiplicativo della riduzione di dimensione e il fattore moltiplicativo dell'aumento di dimensione. Di solito si raddoppia quando è pieno e si divide la dimensione per tre quando è occupato per un terzo.
In generale i fattori moltiplicativi vanno tarati in base al problema, non c'è una soluzione unica generale.

Capitolo 3

Liste

3.1 Introduzione

Definiamo una lista come una collezione di dati in cui è presente una relazione di posizione, cioè ogni dato ha un precedente e un successivo, fatta eccezione per il primo e l'ultimo elemento.

Importante notare che una lista è un tipo di dato astratto (ADT) che può essere implementata concretamente sfruttando altre strutture dati, per esempio possiamo usare un'array o una catena concatenata o doppiamente concatenata. In base alla struttura dati scelta variano le prestazioni temporali. In Java è semplice riconoscere le ADT dalle implementazioni concrete: le interfacce rappresentano le ADT; le classi che implementano l'interfaccia pubblica rappresentano l'implementazione.

In una lista ogni elemento può essere identificato univocamente in diversi modi:

- mediante indice: un numero compreso tra 0 e `size() - 1` (intesa come dimensione logica della lista). L'indice di un elemento è definito come il numero di elementi che lo precedono nella sequenza;
- mediante rango: un numero compreso tra 1 e `size()`. Questo metodo viene usato nei linguaggi che hanno gli indici degli array che partono da 1.
- mediante una rappresentazione astratta della posizione.

3.2 Lista con indice o con rango

Viene chiamata tradizionalmente vettore.

```
1 public interface IndexList<T> extends Container {  
2     T get(int index);  
3     T set(int index, T element);  
4     T remove(int index);  
5     void add(int index, T element);  
6 }
```

Tutti i metodi lanciano `OutOfBoundsException`.

- `get` ritorna l'elemento alla posizione `0 <= index < size()`;
- `set` modifica l'elemento alla posizione `0 <= index < size()`, torna l'elemento che era presente nella posizione;

- `add`: inserisce un elemento nella lista in una data posizione, `index` deve essere compreso (non strettamente) tra 0 e `size()`. Se l'elemento viene inserito all'interno della lista, bisogna far traslare tutti gli elementi di una posizione;
- `remove`: l'indice deve essere compreso strettamente tra 0 e `size()`. Rimuovere un elemento implica far traslare tutti gli altri per "chiudere il buco". Torna l'oggetto rimosso.

Prestazioni:

Realizziamo una `IndexList` mediante una struttura ad accesso casuale, cioè un array. In questo modo otteniamo che i metodi `get` e `set` hanno prestazioni $\Theta(1)$, se usassimo una struttura ad accesso sequenziale come una catena, otterremo prestazioni $\Theta(n)$.

I metodi `add` e `remove`: nel caso peggiore (n spostamenti) e nel caso medio ($\frac{n}{2}$ spostamenti) sono $\Theta(n)$; nel caso migliore, utilizzando un array riempito in parte con ridimensionamento in piccolo e in grande, si ottengono prestazioni $\Omega(1)$ (analisi ammortizzata).

3.3 Lista con posizione astratta

In questo caso usiamo una catena (o doppia catena) per realizzarla. Per rappresentare il concetto di posizione dobbiamo trovare un sostituto all'indice, ci serve un oggetto più complesso.

```

1  public interface Position<T> { // rappresenta il concetto astratto di
2      T element(); // restituisce l'elemento contenuto alla posizione
3  }
```

`Position<T>` rappresenta l'analogo di `int` per le liste con indice.

Per rappresentare concretamente il concetto di posizione possiamo usare i nodi delle catene: hanno esattamente un successivo e un precedente (esclusi gli estremi). In Java esistono le classi `SLLNode` (simple linked list node) e `DLLNode` (double linked list node) che realizzano questa interfaccia.

Possiamo definire l'ADT `PositionalList`:

```

1  public interface PositionalList<T> extends Container {
2      // metodi di navigazione tra le posizioni
3      Position<T> first() throws EmptyListException;
4      Position<T> last() throws EmptyListException;
5      Position<T> prev(Position<T> p) throws InvalidPositionException,
6          BoundaryViolationException;
7      Position<T> next(Position<T> p) throws InvalidPositionException,
8          BoundaryViolationException;
9      // metodi di modifica del contenuto della lista
10     void addFirst(T element);
11     void addLast(T element);
12     void addBefore(Position<T> p, T element) throws InvalidPositionException;
13     void addAfter(Position<T> p, T element) throws InvalidPositionException;
14     T set(Position<T> p, T element) throws InvalidPositionException;
15     T remove(Position<T> p) throws InvalidPositionException;
16 }
```

Analizziamo i metodi:

- `prev()`, `next()`: permettono di navigare nella lista. In base al tipo di nodo che scelgo di utilizzare variano le prestazioni, per esempio se utilizzo `SLLNode`, il metodo `prev()` sarà molto lento;
- `first()`, `last()`: permettono di accedere alla lista, senza questi metodi non potrei accedere banalmente a nessun elemento in quanto non posso creare oggetti `Position` (è un'interfaccia) e non posso creare oggetti `*Node` (classe privata).
- `set(p, e)`: sostituisce l'elemento presente nella posizione passata (torna il valore) con l'elemento passato;
- `remove(p)`: rimuove l'elemento alla posizione passata dalla lista, torna il valore che viene rimosso;
- `addBefore(p, e)`, `addAfter(p, e)`: aggiunge un nuovo elemento prima/dopo della posizione indicata;
- `addFirst(p, e)`, `addLast(p, e)`: aggiunge un elemento in prima/ultima posizione.

Prestazioni:

Non abbiamo definito un metodo `get` in quanto `Position` di occupa si restituire l'elemento in una posizione. Potevamo decidere di rimpiazzare nello stesso modo anche il metodo `set` di `PositionalList` oppure potevamo dichiarare l'interfaccia `Position` priva di metodi (serve soltanto a introdurre un nome a un ADT). In quest ultimo caso bisognerebbe definire il metodo `get` in `PositionalList`.

Tutti i metodi di `PositionalList` sono $\Theta(1)$ se implemento l'interfaccia con una catena doppia, se scelgo una catena semplice i metodi che necessitano riferimenti all'indietro diventano $O(n)$. Il vantaggio della catena semplice è che occupa meno spazio (non a livello asintotico, cambia il coefficiente moltiplicativo).

3.4 Iteratore di una lista

Un iteratore è un oggetto che ispeziona gli elementi di una lista. In generale gli iteratori vengono implementati per scansionare ADT di tipo sequenziale, cioè in cui è definita una relazione d'ordine tra successivo e precedente.

Sfruttando un iteratore possiamo utilizzare algoritmi di ricerca sequenziale, ricerca massimo/minimo, calcolo elemento medio, ecc.

Un iteratore I è l'insieme di:

- una lista L su cui opera;
- un riferimento a un elemento corrente c di L ;
- una modalità per spostarsi all'elemento successivo di c di L , facendolo diventare il nuovo elemento corrente.

L'implementazione concreta di `Iterator` dipende dal tipo di lista che vogliamo rendere iterabile, il comportamento è generico.

```

1 public interface Iterator<T> {
2     T next() throws NoSuchElementException; // restituisce elemento
      successivo a quello corrente
3     boolean hasNext(); // ritorna true se esiste un elemento successivo a
      quello corrente
4 }

```

Per rendere una lista iterabile dobbiamo implementare l'interfaccia `Iterable`.

```
1 public interface Iterable<T> { // java.lang.Iterable
2     java.util.Iterator<T> iterator(); // restituisce un esemplare di una
        specifica classe (interna privata) che realizza l'interfaccia
        Iterator<T>
3 }
```

Ci potremmo chiedere se, in una lista scansionabile, è meglio tornare un riferimento `Iterable` o un riferimento `Iterator`. La differenza è che con un `Iterator` posso eseguire una sola scansione, con `Iterable` quante ne voglio.

Progettare un iteratore:

Per prima cosa precisiamo che un iteratore può essere implementato anche con una classe esterna, ma è più efficiente (non a livello asintotico) implementarlo con una classe interna (accesso diretto alle variabili di esemplare).

Nel progetto di un iteratore ci sono due strategie:

1. snapshot iterator: l'oggetto iteratore agisce su una copia della lista. Necessitiamo uno spazio aggiuntivo $\Theta(n)$, le prestazioni del metodo `iterator()` sono $\Theta(n)$. Se modifico la lista non l'iteratore non lo vede;
2. lazy iterator: l'iteratore agisce direttamente sulla lista. Non richiede spazio aggiuntivo e `iterator()` è $\Theta(1)$. Può portare a dei problemi se modifico la lista mentre la scansiono.

3.5 IndexList o PositionalList?

Non c'è una risposta generale, dipende molto dal tipo di applicazione che vogliamo realizzare. Se sappiamo a priori che l'applicazione richiederà molti inserimenti allora sarà meglio sfruttare una `Positional list` in quanto abbiamo visto che i metodi di inserimento e rimozione sono $\Theta(1)$, se l'applicazione richiederà più accessi in lettura che inserimenti, allora è meglio optare per una `IndexList`.

Capitolo 4

Alberi

4.1 Introduzione

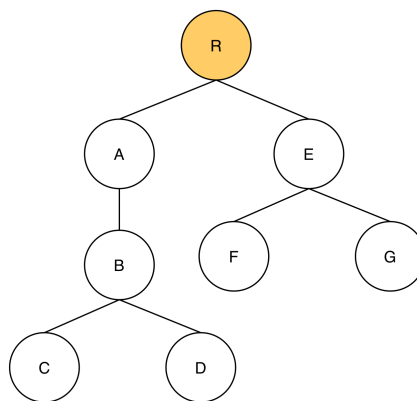


Figura 4.1: Un albero

Definiamo un albero T come un modello astratto per rappresentare una struttura gerarchica.

Carrellata di definizioni e termini che useremo:

- **definizione di albero:** Un albero T è costituito da un insieme di nodi tra i quali esiste una relazione gerarchica di tipo genitore figlio rappresentata mediante rami definita nel seguente modo:
 - **radice:** se un albero T non è vuoto, uno e uno solo dei suoi nodi non ha genitore e viene chiamato radice di T ;
 - **genitore/figli:** ogni nodo diverso dalla radice ha un unico genitore p e i nodi aventi p come genitore sono chiamati figli;
 - **vincolo di connessione:** partendo da qualsiasi figlio e seguendo rami nella direzione che va da un figlio verso il suo genitore, si deve giungere alla radice;
- **ramo:** un ramo dell'albero T è una coppia $\{u, v\}$ di nodi di T tale che u sia il genitore di v o viceversa;
- **definizione albero ricorsiva:** un albero t è vuoto, oppure è costituito da un nodo r , detto radice di T e da un insieme di alberi non vuoti, le cui radici sono i figli di r ;
- **fratelli:** nodi figli dello stesso nodo. Un nodo avente almeno un figlio si dice **interno**, altrimenti si chiama esterno o **foglia**;

- **antenato/discendente:** un nodo a è un antenato del nodo n se e solo se: $a \equiv n$ oppure a è un antenato del genitore p di n . Un nodo d è un discendente del nodo n se e solo se n è un antenato di d . Quindi ogni nodo è antenato e discendente di se stesso;
- **sottoalbero:** dato l'albero T , il suo sottoalbero avente come radice $v \in T$ è l'albero costituito da tutti e soli i discendenti di v ;
- **albero ordinato:** Un albero è ordinato se, per ogni nodo, esiste una relazione posizionale tra i suoi figli. Graficamente i nodi figli di uno stesso genitore vengono disposti da sinistra a destra in base all'ordinamento;
- **grado:** il grado di un nodo è il numero dei suoi figli. Il grado di un albero è il massimo grado dei suoi nodi;
- **dimensione:** la dimensione di un albero è uguale al numero dei suoi nodi. Oppure possiamo dire che la dimensione è data da uno più la somma delle dimensioni degli (eventuali) sottoalberi che hanno come radice un figlio della radice;
- **percorso:** un percorso nell'albero T è una sequenza posizionale (una lista) di nodi di T tale che ogni coppia di nodi consecutivi della sequenza sia un ramo di T ;
- **distanza:** il più breve percorso che collega due nodi appartenenti ad uno stesso albero;
- **livello:** insieme di nodi che hanno la stessa profondità.

4.2 L'interfaccia Tree

Dividiamo l'interfaccia in due parti:

1. gestione dell'albero come container, gestione della relazione gerarchica: metodi che permettono di verificare se la struttura è vuota e metodi che permettono di navigare e ispezionare;
2. modifica della relazione tra i nodi: metodi che permettono di inserire nuovi nodi, (eventualmente) rimuovere nodi esterni, (equivalentemente) modificare le relazioni parentali di un nodo.

I metodi di modifica dipendono strettamente dall'applicazione che vogliamo realizzare, quindi quello che si fa in pratica è definire un'interfaccia generica `Tree` che dichiara i metodi del punto 1, si estende l'interfaccia `Tree` con una nuova interfaccia in cui si dichiarano i metodi di modifica.

Quindi l'interfaccia `Tree` non è utilizzabile direttamente in quanto permette solamente di accedere ad alberi vuoti.

```

1  public interface Tree<T> extends Container, Iterable<T> {
2      Position<T> root() throws EmptyTreeException;
3      Position<T> parent(Position<T> v) throws InvalidPositionException,
        BoundaryViolationException;
4      Iterable<Position<T>> children(Position<T> v) throws
        InvalidPositionException;
5      int numChildren(Position<T> v) throws InvalidPositionException;
6      boolean isRoot(Position<T> v) throws InvalidPositionException;
7      boolean isInternal(Position<T> v) throws InvalidPositionException;
8      boolean isExternal(Position<T> v) throws InvalidPositionException;
9      // Iterator<T> iterator(); parte di Iterable
10     Iterable<Position<T>> positions();

```



```

11     T replace(Position<T> v, T element) throws InvalidPositionException;
12     // int size(); parte di Container
13     // boolean isEmpty(); parte di Container
14 }

```

- `root()`: ritorna la radice, permette di ottenere una posizione valida da cui partire per poter navigare l'albero;
- `parent(v)`: fornisce il nodo genitore del noto passato;
- `children(v)`: fornisce una lista (anche vuota) contenente `Position`, cioè le posizioni dei figli del nodo passato come argomento; Una `Iterable` list è una qualsiasi lista che implementa l'interfaccia `Iterable`;
- `numChildren(v)`: restituisce il numero dei figli di un nodo passato come argomento. Per risalire al numero dei figli, il metodo sfrutta `children`. Metodo non necessario ma risparmia fatiche all'utilizzatore;
- `isRoot(v)`, `isExternal`, `isInternal`: metodi non necessari ma sono utili come condizioni per navigare l'albero;
- `iterator()`: restituisce un iteratore che opera su una lista contenente i dati presenti nell'albero;
- `positions()`: restituisce una lista (iterabile) contenente le posizioni dei nodi dell'albero. Questo metodo e il precedente, `linearizzano` l'albero, sono utili nei casi in cui non ci interessa la relazione gerarchia tra i nodi (es: somma di tutti gli elementi contenuti nei nodi);
- `replace(v, elem)`: modifica il dato contenuto in un nodo (non modifica la relazione del nodo).

Prestazioni:

Per realizzare una classe che implementi l'interfaccia `Tree` è conveniente utilizzare una struttura concatenata, la classe dovrà dichiarare due variabili di esempio: un riferimento al nodo radice; una variabile che tenga traccia della dimensione.

Per quanto riguarda il nodo, dobbiamo realizzare una classe che implementa l'interfaccia `position`. La classe (interna alla classe che implementa `Tree`) dovrà avere tre variabili di esempio: un riferimento al nodo `parent`; una linked list di nodi `children`; il dato contenuto nel nodo.

Tutti i metodi che abbiamo dichiarato sono realizzabili attraverso algoritmi che hanno prestazioni $\Theta(1)$, fatta eccezione per i metodi `positions(v)` e `iterator()` che sono $\Theta(n)$ (n = nodi albero) e per il metodo `children(v)` che è $\Theta(g_v)$, dove g_v è il grado di v (questo perché la lista contenente i figli viene clonata, non viene passato un riferimento).

In generale si richiede che una struttura dati abbia una occupazione lineare $\Theta(n)$, dove n indica il numero di nodi. Nel nostro caso questa condizione è verificata, infatti abbiamo: un nodo per ogni dato implica n nodi; due variabili d'esempio per l'albero; le liste dei figli che hanno dimensione totale uguale a $n - 1$ (l'unico nodo che non è figlio è la radice).

Per calcolare i figli presenti in un albero di dimensione n , basta ricordarsi la seguente formula

$$\sum_{v \in T} g_v = \dim(T) - 1 = n - 1 \quad (4.1)$$

4.3 Profondità di un nodo

Profondità: la profondità (depth) d_v di un nodo $v \in T$ è il numero dei suoi antenati propri (numero antenati v escluso). $d_v \geq 0$ e $d_v = 0$ se v è la radice.

Alternativamente possiamo utilizzare una definizione ricorsiva: `if T.isRoot(v) d_v = 0; else d_v = 1 + d_v;`.

Siccome il percorso costituito da tutti e soli gli antenati di un nodo è il più breve che collega il nodo alla radice, possiamo dire che la profondità di tale nodo coincide con la sua distanza dalla radice. Precisiamo che non è sempre vero che la distanza tra due nodi è data dalla somma delle loro profondità, in realtà questa regola vale se e solo se i due nodi appartengono a due sottoalberi della radice differenti.

Profondità massima: in un albero non vuoto di dimensione n , la profondità massima di un nodo è minore di n .

$$\max_{v \in T \neq \emptyset} d_v < n \quad (4.2)$$

Si dimostra banalmente per assurdo.

Calcolare la profondità:

```

1 public static <T> int depth(Tree<T> t, Position<T> v) {
2     if (t == null) throw new IllegalArgumentException();
3     if (t.isRoot(v)) return 0;
4     return 1 + depth(t, t.parent(v)); // ricorsione semplice
5 }
```

Le prestazioni temporali sono date dal numero di invocazioni ricorsive. Questo numero dipende dalla profondità del nodo v , quindi (sfruttando 4.2) si conclude che il metodo ha prestazioni $\Theta(d_v)$ (lo stesso vale per l'occupazione).

4.4 Altezza di un nodo

Altezza di un nodo: l'altezza h_v di un nodo $v \in T$ è così definita (ricorsivamente): `if T.isExternal(v) h_v = 0; else h_v = 1 + max_{c \in T.children(v)} h_c`.

Un nodo ha altezza zero se e solo se è una foglia, la radice è il nodo avente altezza massima e l'altezza di un albero non vuoto è l'altezza della sua radice.

Nota che nodi aventi stessa profondità possono avere altezze differenti. In generale hanno profondità e altezza uguale se l'albero è simmetrico.

Calcolo dell'altezza:

```

1 if T.isExternal(v) return 0
2 return 1 + max_{c \in T.children(v)} height(c)
```

Il metodo calcola l'altezza di un nodo sfruttando la definizione ricorsiva. L'invocazione ricorsiva avviene al massimo una volta per ciascun nodo dell'albero: parte dalla radice e scende verso i figli, non risale mai. Siccome ogni nodo ha un solo genitore, il metodo `height(v)` viene invocato, al più, su ogni nodo.

L'algoritmo `height(v)` richiede un tempo di esecuzione suddiviso in due porzioni: una costante $\Theta(1)$ per controllare se v è una foglia; una proporzionale al numero al grado g_v del nodo, cioè un tempo $\Theta(g_v)$, per l'invocazione di `children(v)` e la conseguente scansione della lista ottenuta.

Il tempo totale richiesto per l'esecuzione di `height(r)` per calcolare l'altezza della radice di un albero di dimensione n è

$$\Theta\left(\sum_{v \in T} (1 + g_v)\right) = \Theta\left(\sum_{v \in T} 1 + \sum_{v \in T} g_v\right) = \Theta(\dim T + (\dim T - 1)) = \Theta(n)$$

Se vogliamo calcolare l'altezza di un nodo diverso dalla radice, si dimostra allo stesso modo che il metodo ha prestazioni $\Theta(\dim(S(v)))$, dove $S(v)$ è il sottoalbero avente come radice v .

4.5 Attraversamenti di alberi

L'attraversamento di un albero è una strategia di ispezione che visita tutti i nodi dell'albero una sola volta. Con visita intendiamo qualsiasi azione che riguardi quel nodo, per esempio la visualizzazione del contenuto.

Esistono diversi attraversamenti di alberi, differiscono per l'ordine in cui visitano i nodi. I più importanti sono:

- attraversamento in pre-ordine o in ordine anticipato: ciascun nodo dell'albero viene visitato prima di tutti i suoi discendenti propri, e dopo tutti i suoi antenati propri;
- attraversamento in post-ordine o in ordine posticipato: ciascun nodo dell'albero viene visitato dopo tutti i suoi discendenti propri, e prima di tutti i suoi antenati propri;
- attraversamento "per livelli" o "in ampiezza": nodi di profondità inferiore vengono visitati prima di nodi con profondità maggiore.

Per gli alberi binari (vedi sezione successiva) vedremo l'attraversamento in ordine simmetrico.

L'algoritmo di navigazione completa di un albero, non compie elaborazioni, queste dipendono dalla singola azione della visita.

4.5.1 Attraversamento in pre-ordine

```

1  preOrderTraversal(T) // innesca la ricorsione
2      if !T.isEmpty()
3          preOrderTraversal(T, T.root()) // lancia la ricorsione
4  preOrderTraversal(T, v) // metodo ricorsivo, attraversa Sv
5      visit(v) // azione di visita, metodo esterno che probabilmente deve
6          ricevere anche T
7      C = T.children(v)
8      for each w ∈ C
          preOrderTraversal(T, w) // ricorsione multipla, le invocazioni
              dipendono dal grado di T

```

Dimostrazione correttezza:

Dimostrazione. Vogliamo dimostrare che ogni nodo dell'albero viene visitato una e una sola volta. Partiamo dimostrando che: "ogni nodo dell'albero viene visitato". Poniamo

$$H \equiv \{y \in T \mid y \text{ non viene mai visitato da } \text{preOrderTraversal}(T, T.\text{root}())\}$$

La tesi è $H = \emptyset$. Per assurdo: $H \neq \emptyset$ quindi $\exists v \in H \mid \text{depth}(v) \leq \text{depth}(z)$. Questa costruzione di H deriva da come funziona l'attraversamento in pre-ordine. Abbiamo due casi possibili:

- v è la radice di T : non è possibile che la radice non venga visitata, è il primo nodo passato al metodo ricorsivo. Assurdo.
- v non è la radice: allora esiste $p = T.parent(v)$. Se $v \in H \Rightarrow p \in H$, ma $p = T.parent(v) \Rightarrow depth(p) = depth(v) - 1 < depth(v)$. Siccome $p \in H$, allora non è vero che $depth(v) < depth(z) \forall z \in H$. Assurdo.

Dimostriamo che: "nessun nodo viene visitato più di una volta". Poniamo

$$K \equiv \{y \in T \mid y \text{ viene visitato più di una volta da } preOrderTraversal(T, T.root())\}$$

La tesi è $K = \emptyset$. Per assurdo: $K \neq \emptyset$ quindi $\exists v \in H \mid depth(v) \leq depth(z)$. Abbiamo due casi possibili:

- v è la radice di T : non è possibile che la radice venga visitata più di una volta, dovrebbe comparire nella lista dei figli di un nodo, ma la radice non ha genitori. Assurdo.
- v non è la radice: allora esiste $p = T.parent(v)$. Se $v \in H \Rightarrow p \in H$, in quanto ogni volta che v viene visitato, significa che è stato invocato $preOrderTraversal(T, v)$, e questo avviene una sola volta durante la scansione della lista dei figli all'interno dell'esecuzione di $preOrderTraversal(T, v)$, che quindi è stato invocato più volte e ognuna di queste invocazioni visita p . Siccome $p \in H$, allora non è vero che $depth(v) < depth(z) \forall z \in H$. Assurdo.

Abbiamo dimostrato che l'algoritmo è di attraversamento. ■

Prestazioni:

Le prestazioni temporali dipendono dal metodo $visit(v)$. Se poniamo che sia $\Theta(1)$, allora l'attraversamento in pre-ordine ha prestazioni $\Theta(n)$, in quanto visita tutti i nodi. Ciò si dimostra facilmente andando ad analizzare il codice eseguito per ogni nodo: visita del nodo $\Theta(1)$; l'esecuzione del ciclo dipende dal grado del nodo, quindi è $\Theta(g_v)$. Sfruttando 4.1 si ottiene:

$$\Theta \left(\sum_{v \in T} (1 + g_v) \right) = \Theta(n)$$

Finché il metodo $visit(v)$ ha prestazioni inferiori o uguali a $\Theta(g_v)$, le prestazioni dell'algoritmo non cambiano.

Per quanto riguarda la memoria: la profondità di ricorsione dell'algoritmo è uguale all'altezza dell'albero, quindi l'occupazione è $\Theta(h)$; bisogna sommare l'occupazione data dall'invocazione di $T.children(v)$ dei metodi ricorsivi attivi sul runtime stack, nel caso pessimo è $O(n)$ (quando tutte le ricorsioni sono attive). Infine, contanto il fatto che solitamente un'operazione di visita ha un'occupazione $\Theta(1)$, si ottiene che lo spazio aggiuntivo richiesto è $\Omega(h)$ e $O(n)$.

Generalmente l'azione di visita implementata da $visit(v)$, segue uno di questi schemi:

- modifica in qualche modo il nodo e non produce un effetto globale dovuto all'attraversamento (es: scrivere un dato nel nodo);
- l'attraversamento restituisce una variabile che rappresenta un risultato globale, elaborato componendo il risultato prodotto dalle azioni di visita (es: conteggio foglie di un albero).

Vediamo due esempi di algoritmi che sfruttano l'attraversamento in pre-ordine:

```

1 // Algoritmo che scrive la depth in ciascun nodo
2 public static void setDepth(Tree<Integer> t) { // metodo stub
3     if (t != null && !t.isEmpty())
4         setDepth(t, t.root(), 0); // depth radice = 0
5 }
6 public static void setDepth(Tree<Integer> t, Position<Integer> v, int
7     depthV) {
8     t.replace(v, depthV); // visita
9     for (Position<Integer> x : t.children(v))
10         setDepth(t, x, depthV+1)
11 }

```

```

1 // Algoritmo che conta le foglie (similmente si contano nodi interni o nodi
2   totali)
3 public static <T> int countLeaves(Tree<T> t) {
4     if (t == null || t.isEmpty()) return 0;
5     return countLeaves(t, t.root());
6 }
7 private static <T> int countLeaves(Tree<T> t, Position<T> v) {
8     if (t.isExternal(v)) return 1; // azione di visita di una foglia
9     // azione di visita di un nodo interno
10    int count = 0;
11    for (Position<T> x : T.children(v))
12        count += countLeaves(t, x);
13    return count; // foglie nel sottoalbero con radice v
14 }

```

Possiamo implementare il metodo `positions()` dell'interfaccia `Tree` sfruttando l'attraversamento in pre-ordine.

```

1 import java.util.ArrayList;
2 public class XyzTree<T> implements Tree<T> {
3     public Iterable<Position<T>> positions() { // oppure un altro tipo di lista
4         ArrayList<Position<T>> list = new ArrayList<Position<T>>(); // vuota
5         if (isEmpty()) return list; // lista vuota
6         preorderAppendPosition(list, root());
7         list.trimToSize(); // evita spreco di spazio
8         return list;
9     }
10    private void preorderAppendPosition(ArrayList<Position<T>> list, Position<T>
11        v) {
12        list.add(v); // azione di visita  $\Theta(1)$  in media
13        for (Position<T> c : children(v))
14            preorderAppendPosition(list, c);
15    }

```

4.5.2 Attraversamento in post-ordine

```

1 postOrderTraversal(T)
2     if !T.isEmpty()
3         postOrderTraversal(T, T.root())
4 postOrderTraversal(T, v)
5     C = T.children(v)

```

```

6   for each w ∈ C
7       postOrderTraversal(T, w)
8   visit(T, v)

```

La dimostrazione di correttezza, le prestazioni dell'algoritmo e lo spazio occupato in memoria, sono del tutto analoghe a quelle viste per l'attraversamento in pre-ordine.

Mentre l'attraversamento in pre-ordine è utile quando dobbiamo effettuare delle elaborazioni che dipendono da quelle analoghe già svolte sui genitori di un nodo (es: scrivere la profondità), l'attraversamento in post-ordine è utile quando dobbiamo effettuare elaborazioni che dipendono da quelle analoghe già svolte sui figli di un nodo. Per esempio, vediamo l'algoritmo che scrive in un nodo la sua rispettiva altezza.

```

1   public static void setHeight(Tree<Integer> t) {
2       if (t != null && !t.isEmpty())
3           setHeight(t, t.root());
4   }
5   private static int setHeight(Tree<Integer> t, Position<Integer> v) {
6       int height = 0; // foglia
7       for (Position<Integer> x : t.children(v))
8           int h = setHeight(t, x);
9           if (h + 1 > height)
10              height = h + 1;
11   }
12   t.replace(v, height); // visita
13   return height;
14 }

```

In questo modo otteniamo prestazioni ottime, se avessimo scritto lo stesso algoritmo usando un pre-attraversamento, avremmo ottenuto prestazioni pessime.

4.5.3 Attraversamento per livelli o in ampiezza

L'insieme di nodi di un albero che hanno la stessa profondità formano un livello. I livelli sono numerati progressivamente partendo dall'alto, la radice è l'unico nodo che appartiene al livello 0.

```

1   levelOrderTraversal(Tree T):
2       if T.isEmpty()
3           return
4       q = new Queue() // creo una coda
5       q.enqueue(T.root()) // inserisco la radice (livello 0)
6       while !q.isEmpty()
7           v = q.dequeue() // man mano estraggo il nodo da visitare
8           visit(T, v) // visita
9           for each f ∈ T.children(v)
10              q.enqueue(f) // accodo tutti i figli del nodo

```

Per implementare la coda è conveniente utilizzare una lista concatenata in quanto avvengono molti inserimenti e rimozioni.

Prestazioni:

Si osserva che: ogni nodo viene aggiunto alla coda una e una sola volta, quindi $\Theta(n)$; ciascun nodo viene estratto una sola volta, quindi $\Theta(n)$; per ciascun nodo viene creata e scandita la

lista dei figli in un tempo $\Theta(n)$. Possiamo concludere che se l'azione di visita è $\Theta(1)$, allora l'algoritmo ha prestazioni temporali $\Theta(n)$.

4.6 Alberi binari

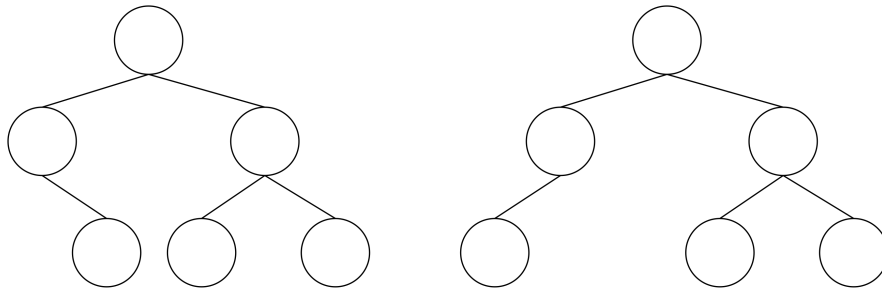


Figura 4.2: Due alberi binari distinti

Gli alberi binari sono alberi ordinati di grado due con una particolarità: un figlio si può chiamare sinistro o destro. Se il nodo possiede due figli, allora il sinistro precede il destro, se un nodo ha un figlio può avere solo il destro o solo il sinistro.

Se non esiste un nodo che ha un solo figlio, allora l'albero binario si dice proprio o pieno, in caso contrario si dice improprio (si estende il termine anche ai singoli nodi).

Chiamiamo sottoalbero sinistro e sottoalbero destro i sottoalberi che hanno come radice rispettivamente il figlio sinistro e il figlio destro della radice. Per facilitare le dimostrazioni di questo tipo di alberi, facciamo una piccola modifica alla definizione di sottoalbero che abbiamo dato in precedenza: poniamo che ciascun nodo abbia sempre entrambi i suoi possibili sottoalberi (sinistro e destro), con la convenzione di ritenere vuoto un albero mancante.

Proprietà:

- il livello d ha al massimo 2^d nodi;
- se l'albero binario è non vuoto:
 - $1 \leq n_e \leq 2^h$;
 - $h \leq n_i \leq 2^h - 1$;
 - $h + 1 \leq n \leq 2^{h+1} - 1$;
 - $\log_2(n + 1) - 1 \leq h \leq n - 1$.
- Se l'albero binario è non vuoto e proprio:
 - $h + 1 \leq n_e \leq 2^h$;
 - $h \leq n_i \leq 2^h - 1$;
 - $2h + 1 \leq n \leq 2^{h+1} - 1$;
 - $\log_2(n + 1) - 1 \leq h \leq (n - 1)/2$;
 - $n_e = n_i + 1$ (i nodi totali n sono dispari).

4.6.1 Attraversamento in ordine simmetrico

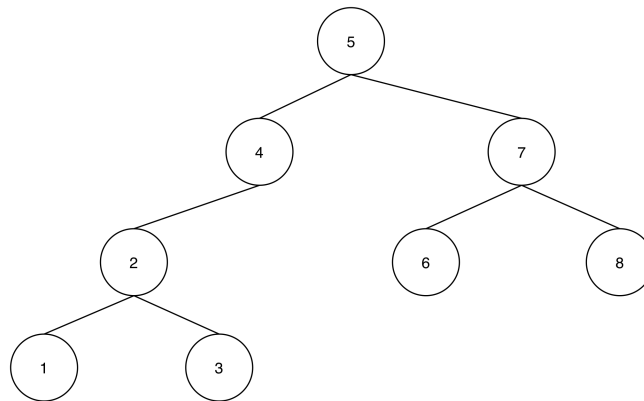


Figura 4.3: Ordine di visita dell'algoritmo: ordine di accesso ai nodi

Nell'attraversamento in ordine simmetrico, viene visitato prima il sottoalbero sinistro di ogni nodo, poi il sottoalbero destro di ogni nodo.

```

1  inorderTraversal( $T \neq \emptyset$ ,  $v \in T$ )
2      if  $v$  ha il figlio sinistro  $u \in T$ 
3          inorderTraversal( $T, u$ )
4      visit( $v$ )
5      if  $v$  ha il figlio destro  $w \in T$ 
6          inorderTraversal( $T, w$ )
  
```

Prestazioni:

Come per gli altri attraversamenti, anche l'attraversamento in ordine simmetrico viene eseguito in un tempo $\Theta(n)$, quando l'azione di visita è $\Theta(1)$ oppure $O(g_v)$.

4.6.2 L'interfaccia BinaryTree. Due possibili implementazioni

```

1  public interface BinaryTree<T> extends Tree<T> {
2      Position<T> left(Position<T> v) throws InvalidPositionException,
3          BoundaryViolationException;
4      Position<T> right(Position<T> v) throws InvalidPositionException,
5          BoundaryViolationException;
6      boolean hasLeft(Position<T> v) throws InvalidPositionException;
7      boolean hasRight(Position<T> v) throws InvalidPositionException;
8  }
  
```

Realizzazione mediante struttura concatenata:

Per realizzare l'interfaccia solitamente si utilizza una struttura a nodi concatenati. La classe nodo che implementa l'interfaccia `Position` è diversa dalla classe nodo per alberi generici, in particolare, nel caso di alberi binari, non ci serve istanziare una lista contenente i figli del nodo, ci basta dichiarare una variabile `left` per il riferimento al figlio sinistro e una variabile `right` per il riferimento figlio destro. Se uno di questi è assente, il valore viene impostato a `null`.

Vediamo un esempio di implementazione di `BinaryTree`:

```

1  public class LinkedBinaryTree<T> implements BinaryTree<T> {
2      private int size;
  
```



```

3     private BTreeNode<T> root;
4
5     public LinkedBinaryTree() { root = null; size = 0; } // costruttore
6
7     private BTreeNode<T> checkPosition(Position<T> v) throws
8         InvalidPositionException {...} // controllo sulla posizione passata
9
10    public Position<T> addRoot(T elem) throws NonEmptyTreeException {...} //
11        posso inserire una sola radice
12    public Position<T> addLeft(Position<T> v, T elem) throws
13        InvalidPositionException {...} // aggiunge un nuovo nodo come figlio
14        sinistro di v
15    public Position<T> addRight(Position<T> v, T elem) throws
16        InvalidPositionException {...} // aggiunge un nuovo nodo come figlio
17        destro di v
18    public Position<T> sibling(Position<T> v) throws
19        InvalidPositionException, BoundaryViolationException {...} // navigo
20        lateralmente
21    public T remove(Position<T> v) throws InvalidPositionException {...}
22
23    public boolean hasLeft(Position<T> v) {...}
24    public boolean hasRight(Position<T> v) {...}
25
26    private static class BTreeNode<T> implements Position<T> {
27        private T element;
28        private BTreeNode<T> parent;
29        private BTreeNode<T> left; // non serve una lista
30        private BTreeNode<T> right; // di figli!
31        ... // metodi get/set per tutte le variabili di esemplare (e metodo
32            element())
33    }

```

Il metodo `remove(v)` può rimuovere il nodo `v` solo se ha al massimo un figlio. Se il nodo è una foglia non ci sono problemi; se il nodo ha un unico figlio, questo prende il posto del suo genitore, cioè diventa figlio di suo nonno (se `v` è la radice, diventa la nuova radice); se il nodo ha due figli viene lanciata un'eccezione, non sappiamo chi sostituisce il nodo rimosso.

Prestazioni:

Valgono le stesse considerazioni fatte per gli alberi generici, tranne per: `children(v)`, è $\Theta(1)$ in quanto i figli sono al massimo due; i metodi modificatori visti in `LinkedBinaryTree` sono realizzabili con complessità $\Theta(1)$.

Realizzazione mediante un array:

Si definisce una funzione $p(v)$ che assegna un indice a ogni nodo `v` di un albero binario `T`:

- se `v` è la radice, allora $p(v) = 1$;
- se `v` è il figlio sinistro di `u`, allora $p(v) = 2p(u)$;
- se `v` è il figlio destro di `u`, allora $p(v) = 2p(u) + 1$;

In questo modo vale la regola $\forall v \neq T.root() \ p(T.parent(v)) = \lfloor p(v)/2 \rfloor$ (parte intera), quindi possiamo calcolare la posizione dei parenti prossimi di un nodo in un tempo $\Theta(1)$. Se a un nodo manca un figlio (o entrambi) viene lasciata una posizione vuota nell'array.

Si può notare che le operazioni di calcolo per risalire alla posizione dei genitori/figli di un nodo sono estremamente rapide: il processore usa degli shift per moltiplicare i numeri e utilizza una OR bit a bit per ottenere il numero dispari che identifica il figlio destro.

Si può progettare una classe `ArrayListBinaryTree` che implementi `BinaryTree` in cui i metodi hanno le stesse prestazioni asintotiche, migliorano però le prestazioni puntuali. Come contro ha lo spreco di memoria: nel caso pessimo (albero solo nodi destri per ogni livello) l'array ha dimensione 2^{h+1} , quindi l'occupazione è $O(2^h)$, nel caso pessimo $O(2^n)$ (visto che $h \leq n - 1$).

Vedremo che questo tipo di implementazione è utile per alberi triangolari completi (alberi ben diversi dal caso limite) (capitolo successivo). Per questi alberi l'occupazione sarà $\Theta(n)$ (indice massimo array uguale a n) e di conseguenza si otterrà un risparmio di memoria specifica (non asintotica) visto che un singolo elemento di un array occupa meno di un'istanza di un nodo.

Per poter realizzare la classe `ArrayListBinaryTree` dobbiamo considerare due dettagli implementativi fondamentali: la scelta del nodo che implementa l'interfaccia `Position`; il metodo `remove(v)`. Per la posizione ci basta definire una classe interna `ArrayBTreeNode` che contenga un riferimento `element` al dato contenuto e un indice che individui la posizione del nodo nell'array. Il metodo `remove(v)` può diventare assai complicato da gestire (a causa di cambi di indice) e non è possibile renderlo $\Theta(1)$, quindi possiamo scegliere di non implementarlo o di farlo e specificare che le prestazioni non sono costanti.

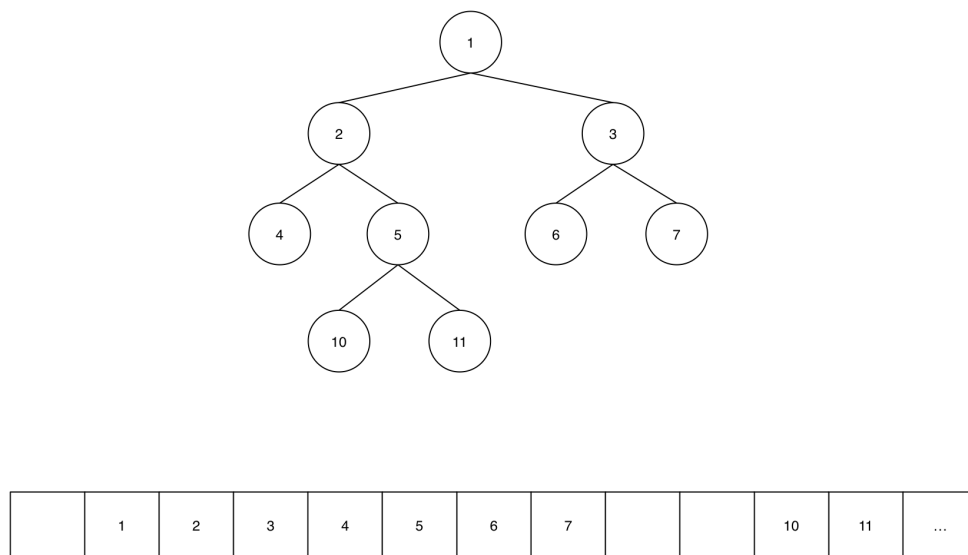


Figura 4.4: Posizione dei nodi all'interno dell'array

Capitolo 5

Code prioritarie

5.1 Relazioni d'ordine

Tutte le ADT che abbiamo visto nei precedenti capitoli gestivano dati senza fare confronti tra di essi, gestivano solamente le relazioni posizionali. Le code prioritarie (priority queue, PQ) sono un esempio di ADT in cui gestiamo le relazioni d'ordine tra gli elementi contenuti.

Una **relazione d'ordine** definita in un insieme S è una funzione $f : S \times S \rightarrow B \equiv \{\text{true}, \text{false}\}$ aventi le seguenti proprietà:

- riflessiva: $\forall a \in S \quad a \leq a$;
- transitiva: $\forall a, b, c \in S \quad a \leq b, b \leq c \Rightarrow a \leq c$;
- antisimmetrica: $\forall a, b \in S \quad a \leq b, b \leq a \Rightarrow a = b$;
- la notazione $a \leq b$ con $a, b \in S$ significa che $f(a, b) = \text{true}$. (Nota che $\leq$ è solo il simbolo che adottiamo).

Una **relazione d'ordine** si dice **totale** per l'insieme S se vale anche la seguente proprietà:

- totalità: $\forall a, b \in S \quad a \leq b$ oppure $b \leq a$ (oppure entrambe). Cioè qualunque coppia di elementi dell'insieme dev'essere confrontabile.

Una relazione d'ordine non totale si dice anche **parziale**.

Dato un insieme S totalmente ordinato (secondo una determinata relazione d'ordine totale f): data una qualunque coppia, è possibile determinare quale sia l'elemento maggiore o minore, quindi gli elementi si dicono confrontabili mediante f ; in qualunque sottoinsieme finito e non vuoto esiste l'elemento massimo e l'elemento minimo (secondo f); gli elementi possono essere disposti in una relazione lineare di tipo precedente/successivo secondo f .

In questo corso supponiamo che, in una relazione d'ordine, ciascun confronto si possa fare in un tempo $\Theta(1)$ in funzione della dimensione dell'insieme S , cioè in un tempo che non dipende dalla dimensione dell'insieme S (ma può dipendere dalla dimensione dei dati). In generale non è sempre così.

5.1.1 Definire relazioni d'ordine in Java

Esistono due approcci: usare un ordinamento intrinseco delle chiavi (es: `java.lang.Comparable<T>`); usare un ordinamento esterno alle chiavi (es: `java.util.Comparator<T>`).

Il problema del primo approccio è che usando la proprietà `Comparable` delle chiavi, non riusciamo a usare un criterio di ordinamento diverso da quello previsto dal progettista della classe. Per esempio usiamo delle stringhe come chiavi, l'unico ordine di priorità realizzabile è

quello indotto dal criterio di ordinamento lessicografico. Non possiamo progettare una coda prioritaria che gestisce le stringhe per esempio in base alla lunghezza.

Per questo motivo utilizziamo l'interfaccia `Comparator`.

```
1 package java.util;
2 public interface Comparator<T> {
3     int compare(T obj1, T obj2);
4 }
```

La classe che implementa `Comparator` diventa un comparatore esterno con l'unico compito di confrontare (in base a come implementiamo il metodo) due oggetti dello stesso tipo.

Per esempio:

```
1 class ByLengthStringComp implements java.util.Comparator<String> {
2     public int compare(String s1, String s2) {
3         if (s1 == null || s2 == null) throw new ClassCastException();
4         if (s1.length() == s2.length()) return s1.length() - s2.length();
5     }
6 }
```

5.2 Introduzione alle code prioritarie

Una coda prioritaria è un contenitore che memorizza elementi associati a un livello di priorità o precedenza.

In una coda prioritaria possiamo eseguire le seguenti operazioni:

- inserire nuovi elementi con specificato un livello di priorità;
- estrarre e/o ispezionare l'elemento avente priorità massima (si parla di `maxPQ` e `minPQ`);

Due esempi di code prioritarie sono: le code (`minPQ`); le pile (`maxPQ`). La priorità di ciascun elemento è il suo istante di inserimento.

Nel gergo gli elementi vengono chiamati valori, le priorità vengono chiamate chiavi. Chiavi e valori rappresentano coppie (in Java oggetti).

Per definire una coda prioritaria occorre quindi specificare: i valori ammessi; le chiavi associate ai valori; una relazione d'ordine totale tra le chiavi.

5.3 L'interfaccia `Entry`

L'interfaccia che rappresenta le coppie chiave valore.

```
1 public interface Entry<K,V> {
2     public K getKey();
3     public V getValue();
4     public V setValue(V newVal);
5     // setKey non ammesso
6 }
```

In generale si definiscono nuove interfacce quando esiste almeno più di una realizzazione possibile di classi che realizzano lo stesso comportamento. Questa interfaccia può essere implementata in un solo modo, sarebbe stato meglio definire direttamente una classe senza passare per l'interfaccia. (Il libro la usa).

5.4 L'interfaccia PriorityQueue

```

1 public interface PriorityQueue<K,V> extends Container {
2     Entry<K,V> insert(K key, V value) throws InvalidKeyException;
3     Entry<K,V> min() throws EmptyPriorityQueueException; // ispezione
4     Entry<K,V> removeMin() throws EmptyPriorityQueueException;
5 } // size() restituisce il numero di Entry presenti

```

`insert(k,v)` rifiuta chiavi che non appartengono all'insieme totalmente ordinato delle chiavi definito, rifiuta `null` e verifica che le chiavi rispettino il criterio di ordinamento totale utilizzato dalla coda prioritaria.

Nella realizzazione concreta dovermo specificare un meccanismo per comunicare al contenitore quale sia la relazione d'ordine totale da utilizzare. Solitamente questo avviene nel costruttore. Per esempio:

```

1 // Nella classe che implementa la coda
2 import java.util.Comparator;
3 public class PQWithComparator<K,V> implements PriorityQueue<K,V> {
4     private Comparator<K> comp;
5     public PQWithComparator(Comparator<K> c) { comp = c; }
6     // i metodi della coda confrontano le chiavi usando comp.compare(x,y)
7     //...
8 }
9 // Nel main
10 PriorityQueue<String, Integer> pq = new PQWithComparator<String,
    Integer>(new ByLengthStringComp()); // comparatore creato dall'utente

```

Implementazione tramite lista:

Possiamo scegliere due strategie: lista non ordinata; lista ordinata.

Se usiamo una lista non ordinata:

- `insert`: inserimento in fondo alla lista $\Rightarrow \Theta(1)$;
- `min`: ispezione di tutte le coppie per trovare la chiave minima $\Rightarrow \Theta(n)$;
- `removeMin`: stessa ricerca che effettua `min` più rimozione $\Rightarrow \Theta(n)$.

Se usiamo una lista ordinata (struttura concatenata, array):

- `insert`: ricerca posizione ($O(n)$, $O(\log n)$) e inserimento ($\Theta(1)$, $O(n)$ con eventuale ridimensionamento $\Theta(n)$) $\Rightarrow O(n)$ caso medio, $\Theta(n)$ caso peggiore;
- `min`: chiave minima in prima posizione (in ultima se uso un array) $\Rightarrow \Theta(1)$;
- `removeMin`: come per `min` più rimozione $\Rightarrow \Theta(1)$.

Quindi se usiamo una lista ordinata abbiamo rimozioni veloci, se usiamo una lista non ordinata abbiamo inserimenti veloci.

Esistono delle applicazioni che implementano le code prioritarie in cui prima si fanno (quasi) tutti gli inserimenti e poi (quasi) tutte le rimozioni. In questi casi si può sfruttare una lista non ordinata nella prima fase di inserimento, poi (su comando dell'utente) si passa alla fase delle rimozioni, quindi si riordina la lista ($\Theta(n \log n)$). Complessivamente le prestazioni sono $\Theta(n \log n)$, se non facessimo lo scambio della lista avremmo prestazioni $O(n^2)$ e $\Theta(n^2)$ nel caso peggiore.

5.5 Heap

5.5.1 Definizione di Heap

Prima di definire un heap introduciamo alcune nuove definizioni:

- **essere a sinistra di:** dati due nodi v, w distinti e posti allo stesso livello di un albero binario, diciam che v è a sinistra di w se v precede w nell'attraversamento in ordine simmetrico dell'albero che li contiene (altrimenti v è a destra di w).
- **albero binario completo:** un albero binario è completo se è triangolare, oppure se è triangolare fino al livello $h-1$, poi ha un ultimo livello "compatto a sinistra". Compatto a sinistra significa che: ho un nodo che è il più a sinistra possibile; proseguendo da questo nodo verso destra, appena trovo un buco non posso più trovare nodi nelle posizioni successive; L'inserimento in un albero binario completo deve essere implementato in modo che venga mantenuta la proprietà di completezza;
- **ultimo nodo:** l'ultimo nodo di un albero binario completo è la foglia più a destra tra quelle più profonde.

Si dimostra che in un albero binario completo di dimensione $h > 0$

$$h = \lfloor \log_2 n \rfloor \quad (5.1)$$

quindi (quindi $h \in \Theta(\log n)$).

Heap:

Uno heap è un albero binario completo T che memorizza in ciascun nodi $v \in T$ una entry $e(v) = (k, val)$, composta da una chiave $k = key(v) = e.getKey()$, appartenente a un insieme totalmente ordinabile e da un valore, $val = value(v) = e.getValue()$.

Se soddisfa la seguente proprietà (detta **proprietà di heap**)

$$\forall v \in T, v \neq T.root() \Rightarrow key(parent(v)) \leq key(v)$$

viene chiamato minHeap, altrimenti si tratta di un maxHeap. Quindi nella radice di un minHeap è presente la chiave minima.

5.5.2 L'interfaccia CompleteBinaryTree

L'interfaccia CompleteBinaryTree estende BinaryTree, l'interfaccia che abbiamo visto nel precedente capitolo. Con questa estensione aggiungiamo i metodi che ci consentono di inserire i nodi, in particolare questi metodi dovranno preservare la completezza dell'albero.

```

1 public interface CompleteBinaryTree<T> extends BinaryTree<T> {
2     void add(T element); // inserisce il nuovo ultimo nodo
3     T remove() throws EmptyTreeException; // rimuove l'ultimo nodo
4 }
```

Realizzazione:

Come abbiamo anticipato nel capitolo precedente, la realizzazione migliore si ottiene sfruttando un array, l'occupazione è $\Theta(n)$ in quanto l'albero non contiene buchi. Sfruttando la funzione che assegna gli indici ai nodi, si ottiene che se (e solo se) T è completo i suoi n nodi hanno indici consecutivi nell'intervallo $[1, n]$.

Quindi realizzeremo l'interfaccia CompleteBinaryTree attraverso la classe ArrayCompleteBT.

Prestazioni:

I metodi `add()` e `remove()` sono dei semplici inserimenti e rimozioni in un array riempito in parte (sappiamo già dove si trova l'ultimo nodo), quindi hanno complessità $\Theta(1)$ in media. Se decidessimo di utilizzare una struttura concatenata avremmo un peggioramento delle prestazioni, in particolare entrambi i metodi diventerebbero $\Theta(\log n)$, questo è causato dalla ricerca dell'ultimo nodo (vedi esercizio 30).

L'occupazione in memoria è $\Theta(n)$, l'array ha dimensione $n + 1$ (la posizione 0 non è usata), quindi $\Theta(n)$ (considerando anche i ridimensionamenti).

5.6 Realizzazione tramite Heap

Utilizziamo un heap gestito tramite un array.

Prestazioni:

Il metodo `size()` restituisce la dimensione logica dell'array $\Rightarrow \Theta(1)$; il metodo `min()` restituisce il contenuto della radice (seconda cella) $\Rightarrow \Theta(1)$; i metodi `insert(k,v)` e `removeMin()` dipendono dall'altezza, quindi (per 5.1) $\Rightarrow O(\log n)$ e nel caso pessimo $\Theta(\log n)$.

Questa implementazione è asintoticamente molto migliore rispetto all'implementazione mediante lista.

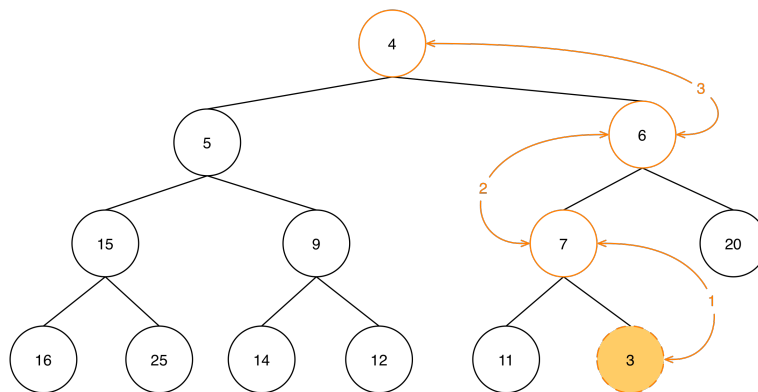


Figura 5.1: Algoritmo up-heap-bubbling

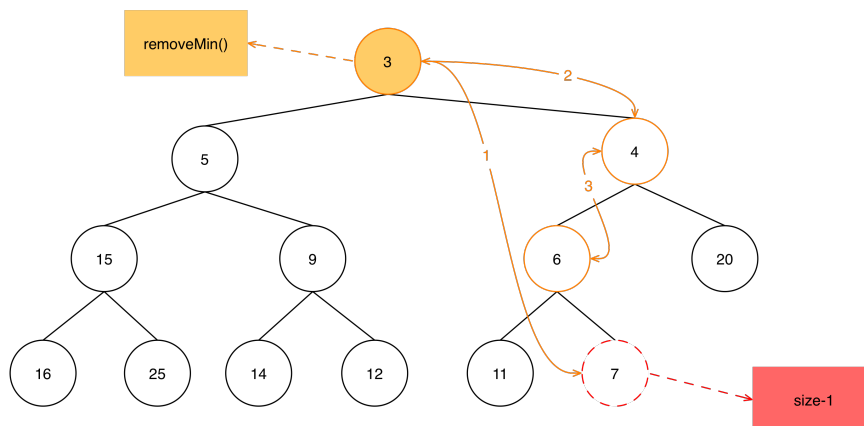


Figura 5.2: Algoritmo down-heap-bubbling

Il metodo `insert(k,v)` è anche chiamato *up-heap-bubbling*: nell'algoritmo si fa risalire il nodo dall'ultima posizione fino (al limite) alla radice. A livello pratico avviene uno scambio di contenuto tra il nodo inserito e il suo genitore, questo finché non viene ripristinata la proprietà di heap.

Il metodo `removeMin()` viene chiamato *down-heap-bubbling*, il funzionamento è simile al metodo `insert(k,v)`, l'unica differenza è che il nodo deve scendere e non risalire. Quando invochiamo il metodo di rimozione avvengono le seguenti operazioni: viene restituito il contenuto della radice; il contenuto dell'ultimo nodo viene copiato nella radice e viene eliminato. A questo punto dobbiamo ripristinare la proprietà di heap. La discesa, a differenza della salita, ha due stadi possibili, si tratta di scegliere uno dei due figli del nodo. Per poter mantenere valida la proprietà di heap, dobbiamo scambiare il contenuto del nodo genitore con il figlio più piccolo (nodo che contiene la chiave minore), ripeto l'operazione finché (al limite) raggiungo le foglie.

5.7 Priority Queue Sort

Si può sfruttare il funzionamento di una priority queue per poter riordinare sul posto gli elementi di una lista. Siccome il miglior modo per poter realizzare una coda prioritaria è quello di sfruttare un heap, questo riordinamento è noto come heap sort.

Realizzare un heap sort non operi sul posto è relativamente semplice: inseriamo gli n elementi di una lista e li inseriamo in una coda prioritaria; svuotiamo la coda prioritaria reinserendo ordinatamente gli n elementi nella lista di partenza. La lista è ora ordinata. Si tratta sostanzialmente di trasformare la lista in uno heap, tale procedura prende il nome di *heapify*.

L'algoritmo di ordinamento heap sort diventa molto più interessante quando opera sul posto. Se la lista da ordinare è una lista con indice, si può evitare di usare esplicitamente un heap, usando invece direttamente la porzione iniziale (di dimensione crescente) di tale lista per realizzare lo heap che serve alla coda prioritaria. Si tratta di trasformare la lista in uno heap, tale processo prende il nome di *heapify*. Le prestazioni asintotiche non cambiano.

Prestazioni:

Pensiamo di voler usare l'heap sort sul posto. Nella prima fase di inserimento nella coda prioritaria, le prestazioni sono $O(n \log n)$: ogni inserimento ha prestazioni $O(\log k)$, dove k è la dimensione (via via crescente) della coda prioritaria al momento dell'inserimento, le operazioni complessive sono in totale $\sum_{i=1}^n \log i = n \log n$. La fase di rimozione ha prestazioni $O(n \log n)$ facendo lo stesso ragionamento.

Quindi l'heap sort ha prestazioni $O(n \log n)$.

Capitolo 6

Mappe

6.1 Introduzione

Una mappa è un contenitore che memorizza valori associati ad una chiave. La funzione principale di una mappa è la ricerca di un valore presente nel contenitore, fornendone la chiave. Si differenzia dalla coda anche dal fatto che le chiavi devono essere univoche, non esistono valori associati alla stessa chiave. In generale non è necessario che le chiavi siano ordinabili, basta che siano confrontabili per uguaglianza, mentre i valori possono essere di qualunque tipo.

6.2 L'interfaccia Map

```
1 public interface Map<K,V> extends Container {
2     V put(K key, V val); // oppure insert
3     V get(K key); // oppure search o find
4     V remove(K key);
5     Iterable<K> keys();
6     Iterable<V> values();
7 } // size() ritorna il numero di coppie
```

In particolare:

- `put(k,v)`: se la chiave non è associata a nessun valore, aggiunge una nuova coppia, altrimenti aggiorna il valore associato. Torna `null` se non è stato avvenuto un inserimento, altrimenti torna il vecchio valore associato alla chiave;
- `get(k)`: ritorna il valore associato alla chiave passata, se la chiave non è presente nella mappa torna `null`;
- `remove(k)`: rimuove la coppia chiave-valore dalla mappa. Se la chiave passata non è presente nella mappa torna `null`;
- `keys()`, `values()`: tornano rispettivamente una lista contenente tutte le chiavi una lista contenente tutti i valori associati. Il vincolo imposto è la corrispondenza tra le posizioni delle due liste.

Implementazione tramite lista:

possiamo realizzare una mappa memorizzando le coppie chiave valore all'interno di una lista (array o lista concatenata). In questo modo i metodi `get(k,v)`, `remove(k)` e `put(k,v)` hanno tutti prestazioni $O(n)$, questo perché necessitano tutti di una ricerca sequenziale. I metodi `keys()` e `values()` sono ovviamente $\Theta(n)$ (non si può fare meglio).

6.2.1 L'interfaccia SortedMap

La mappa aquista di interesse se diventa una mappa ordinata. Il vincolo da imporre riguarda solamente le chiavi: devono esistere una relazione d'ordine.

Nota che il vincolo imposto non implica che la mappa debba obbligatoriamente tenere ordinate le chiavi al suo interno, ovviamente lo si fa per migliorare le prestazioni ma non è un obbligo.

```

1 public interface SortedMap<K,V> extends Map<K,V> {
2     // keys() restituisce una lista ordinata
3 }

```

Implementazione tramite array ordinato:

un array ordinato ci permette di utilizzare l'algoritmo di ricerca per bisezione. In questo modo: `get(k)` diventa $O(\log n)$; `put(k, v)` è $O(\log n)$ se aggiorna un valore, è $O(n)$ se deve inserire (bisogna far spazio alla nuova coppia); `remove(k)` è $O(\log n)$ solo se non rimuove, altrimenti è $O(n)$ (bisogna tappare il buco).

In sostanza, rispetto alla mappa (non ordinata), migliora solo il metodo `get(k)`.

Questo tipo di implementazione è utile per tutte quelle mappe in cui l'operazione di ricerca è molto più frequente delle altre. Per tutti gli altri casi vedremo che utilizzando un particolare tipo di albero si ottengono prestazioni logaritmiche anche per i metodi `remove(k)` e `put(k, v)`.

6.3 Alberi binari di ricerca

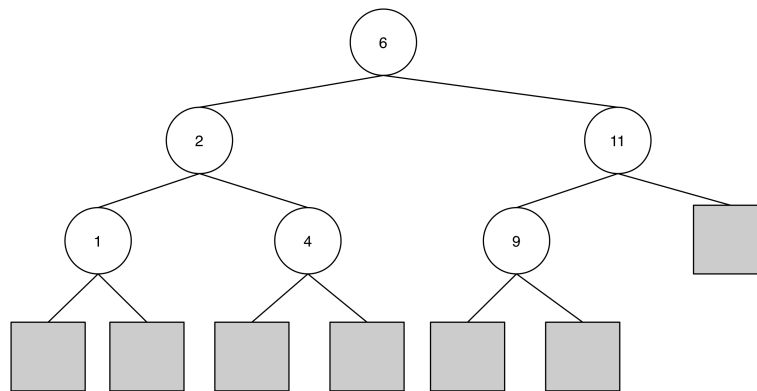


Figura 6.1: Un albero binario di ricerca

Un albero binario di ricerca (Binary Search Tree, BST) è un albero binario proprio, non vuoto, con queste ulteriori caratteristiche:

- ogni nodo interno v contiene un elemento, $x(v)$, appartenente a un insieme totalmente ordinato;
- i nodi esterni non contengono alcun elemento (sono nodi "segnaposto", o placeholder);
- per ogni nodo interno v : gli elementi memorizzati nei nodi interni del sottoalbero sinistro di v sono minori uguali di $x(v)$; gli elementi memorizzati nei nodi interni del sottoalbero destro di v sono maggiori uguali di $x(v)$.

Quindi un BST con placeholder non può essere mai vuoto, deve avere come minimo la radice (contenente `null`) (non contiene elementi ma ha nodi).

Inoltre possiamo notare dalle proprietà che un BST può contenere duplicati nell'albero (non siamo ancora parlando di mappe).

6.3.1 SortedMap con BST

Quelli che abbiamo chiamato elementi nella definizione del BST, ora diventano le coppie della mappa. Il vincolo che dobbiamo imporre al BST è l'unicità degli elementi.

Per far sì che questo tipo di implementazione abbia un senso, l'ulteriore vincolo da porre è quello di avere prestazioni migliori rispetto all'implementazione con array ordinato.

Confronto con array ordinato:

L'occupazione in memoria dell'implementazione tramite array ordinato era $\Theta(n)$, se usiamo un BST otteniamo la stessa complessità: esiste un nodo interno per ogni coppia della mappa, inoltre essendo il BST un albero proprio vale la proprietà $n_E = n_I + 1$. Quindi abbiamo n nodi interni più $n + 1$ nodi esterni, perciò il BST ha dimensione $2n + 1 \Rightarrow \Theta(n)$.

Un'altra importante proprietà che si dimostra dell'albero binario è la seguente: l'attraversamento in ordine simmetrico visita i nodi interni di un BST in ordine di contenuto non decrescente (ossia crescente, in mancanza di duplicati). In questo modo i metodi `keys()` e `values()` hanno complessità $\Theta(n)$.

A questo punto ci rimane da migliorare le prestazioni dei metodi di ricerca, inserimento e rimozione. Il metodo `get(k)` possiamo realizzarlo con il seguente algoritmo:

```

1  SearchInBSTsubTree(T, k, v)
2      if T.isExternal (v) return null // non trovato
3      if k == key(v) return value(v) // trovato
4      if k < key(v) return SearchInBSTsubTree(T, k, T.left(v))
5      return SearchInBSTsubTree(T, k, T.right(v))

```

Questo algoritmo è l'analogo della ricerca per bisezione che opera su array ordinati. L'unico problema è che non è sempre vero che la dimensione del problema viene dimezzata ad ogni passo, questo dipende molto dalla morfologia dell'albero.

Per quanto riguarda le prestazioni temporali: dipendono linearmente dal numero di nodi visitati, che nel caso peggiore equivale all'altezza h dell'albero (si scende sempre verso un figlio). Quindi l'algoritmo ha prestazioni $O(h)$, che nel caso pessimo (millepiedi) è $O(n)$.

Se vogliamo migliorare le prestazioni di `get(k)` dobbiamo porre un vincolo sulla morfologia dell'albero. Vogliamo che l'altezza cresca come una funzione logaritmica. Una proprietà che abbiamo visto degli alberi binari propri ci è d'aiuto: sappiamo che l'altezza minima di un albero binario di dimensione n è $\lceil \log_2(n + 1) - 1 \rceil$. In questo modo le prestazioni di `get(k)` divantano $O(\log n)$ nel caso pessimo, infatti si dimostra che l'algoritmo effettua gli stessi passi dell'algoritmo di ricerca per bisezione.

Costruzione di un BST con prestazioni di `get(k)` ottime:

```

1  * Disponiamo le chiavi in un array A e lo ordiniamo.
2  Se A e' vuoto (caso base) inseriamo nel BST la sola radice/foglia, senza
   contenuto. FINE.
3  * Individuiamo l'elemento "centrale" di A, lo chiamiamo v e lo mettiamo
   nella radice del BST:
4      - Gli elementi del semi-array "sinistro" di A, contenente gli elementi
5  minori di v, verranno memorizzati nel sottoalbero sinistro della radice;
6      - Gli elementi del semi-array "destro" di A, contenente gli elementi
7  maggiori di v, verranno memorizzati nel sottoalbero destro della radice;

```

8

- Ripetiamo ricorsivamente la procedura per i due sottoalberi.
--

Questo algoritmo di costruzione di questo BST effettua le seguenti operazioni: trova l'elemento centrale; inserisce la radice in un nuovo (sotto)albero; individua gli indici di inizio e fine dei due sottoarray.

Tutte operazioni $\Theta(1)$ ripetute per n (coppie mappa) più $n + 1$ (foglie vuote) volte $\Rightarrow$ la costruzione richiede un tempo $\Theta(n)$ se l'array da cui partiamo è ordinato.

Portare tutti i metodi a prestazioni $O(h)$:

Le considerazioni fatte finora non permettono di realizzare una mappa con prestazioni ottime. Prima di passare alla soluzione definitiva, portiamo anche le prestazioni dei metodi di rimozione e inserimento a $O(h)$.

- **put(k,v)**: siccome la ricerca è $O(h)$ dobbiamo solo migliorare l'inserimento. Si può notare che non avviene l'aggiornamento del valore se la chiave non è presente nella mappa; l'algoritmo di ricerca fallisce quando raggiunge una foglia, in particolare questa foglia sarebbe la posizione del nodo se fosse presente nella mappa. Avendo in mano questa posizione, possiamo inserire la coppia in questa foglia. Con questa strategia l'albero può degenerare in un millepiedi, in ogni caso le prestazioni sono $O(h)$ nel caso peggiore.
- **remove(k)**: analizziamo il caso in cui la rimozione avviene. Esistono tre casi:
 1. il nodo v rimosso ha due foglie come figli: elimino le foglie e trasformato v in una foglia. Tempo richiesto costante.
 2. il nodo v ha come figli una foglia e un nodo u interno: elimino la foglia e v , u prende il posto di v . Tempo richiesto costante.
 3. il nodo v ha come figli due nodi interni: dobbiamo mantenere la proprietà del BST, quindi possiamo rimpiazzare v con 2 nodi: il nodo più a sinistra del sottoalbero destro v ; il nodo più a destra del sottoalbero sinistro di v . L'unica operazione costosa è la ricerca dell'elemento che sostituisce v , si può notare facilmente che è $O(h)$ nel caso peggiore.

Come per l'inserimento possiamo avere un caso degenerare che penalizza le prestazioni, ma in ogni caso anche la rimozione diventa $O(h)$ nel caso peggiore.

6.3.2 Alberi AVL

Per poter ottenere delle prestazioni logaritmiche per tutte le operazioni fondamentali di una mappa ordinata, dobbiamo introdurre un nuovo tipo di albero.

Introduciamo una nuova definizione:

- **nodo bilanciato**: un nodo interno v di un albero binario proprio si dice bilanciato in altezza se le altezze dei suoi (due) figli differiscono tra loro al massimo di un'unità (altrimenti si dice **sbilanciato**);
- **albero bilanciato**: un albero è bilanciato se tutti i nodi (interni) sono bilanciati.

Un BST bilanciato si dice albero AVL o semplicemente AVL. Il nome deriva dalle iniziali dei due inventori (1960). La figura 6.1 rappresenta un albero binario di ricerca bilanciato.

Si dimostra che l'altezza di un AVL di dimensione n è $O(\log n)$. Ciò non significa che l'altezza sia minima ma "quasi".

Quindi se si realizza una mappa ordinata con un AVL e si riescono a progettare operazioni **put(k,v)** e **remove(k)** che mantengono l'albero bilanciato, si ottiene **get(k,v)** con prestazioni

$O(\log n)$ nel caso pessimo.

Algoritmi di inserimento e rimozione con prestazioni logaritmiche:

Partiamo dall' **inserimento** $\text{put}(k, v)$. Abbiamo visto un algoritmo che ci permette di ottenere prestazioni $O(h)$. Ci manca solo un piccolo passo: dobbiamo controllare ad ogni inserimento che l'albero sia bilanciato, in caso non lo sia dobbiamo ristrutturarlo.

Diciamo che l'inserimento di un nuovo nodo nell'albero provoca una perturbazione, può causare l'aumento di altezza di uno (o più) dei suoi antenati propri e questo può comportare a uno sbilanciamento dell'albero.

L'algoritmo, dopo aver inserito il nodo, deve andare a verificare che non ci sia nessuno sbilanciamento, deve quindi conoscere le altezze dei nodi. Siccome il calcolo dell'altezza di un nodo è un'operazione costosa, si risolve questo problema aggiungendo una nuova variabile che memorizzi l'altezza all'interno di ogni nodo dell'albero.

L'algoritmo risale l'albero partendo dal nodo inserito, verso la radice, ad ogni passo controlla le altezze (se necessario le aggiorna) e si blocca quando trova il primo nodo sbilanciato. A questo punto vengono individuati tre nodi:

- W = il nodo inserito;
- Z = il primo nodo sbilanciato trovato;
- Y = il nodo figlio di Z con altezza maggiore (si dimostra che questo è antenato di W);
- X = il nodo figlio di Y con altezza maggiore (può coincidere con W).

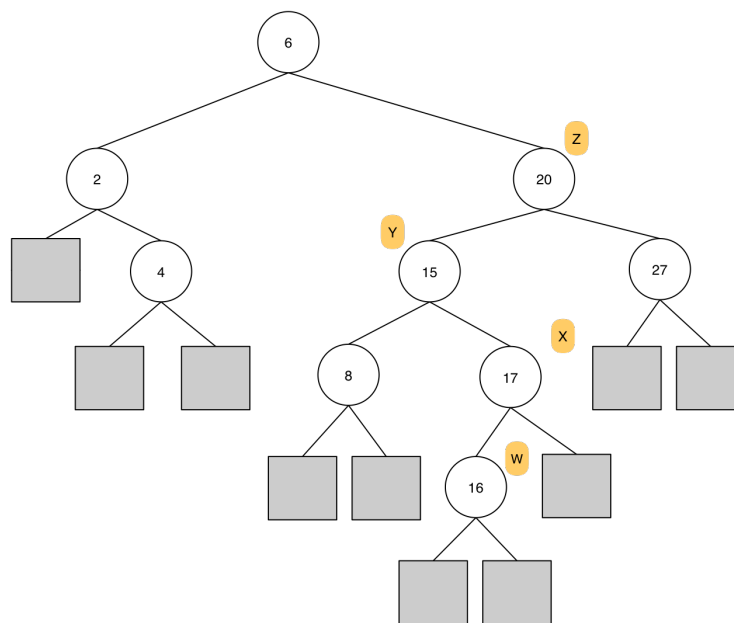


Figura 6.2: Situazione dopo l'inserimento del nodo w

Esistono soltanto quattro configurazioni possibili per i nodi X, Y e Z , inoltre il nuovo nodo interno W appartiene a uno dei due sottoalberi di X (o coincide con X).

Si tratta ora di prendere, tra i nodi X, Y, Z , quello che ha l'elemento intermedio tra i tre (può essere X o Y , mai Z) e lo si fa diventare radice del sottoalbero che aveva radice Z (prende il nome di rotazione), sistemando poi gli altri due nodi di conseguenza, come figli sinistro e destro della nuova radice (nell'unico modo possibile, in relazione al valore dei loro elementi).

Infine ci rimane da attaccare i quattro sottoalberi T^* nei rami liberi, rispettando il loro ordinamento relativo precedente e da correggere di conseguenza le nuove altezze di X, Y, Z (sfruttando le altezze delle radici T^*).

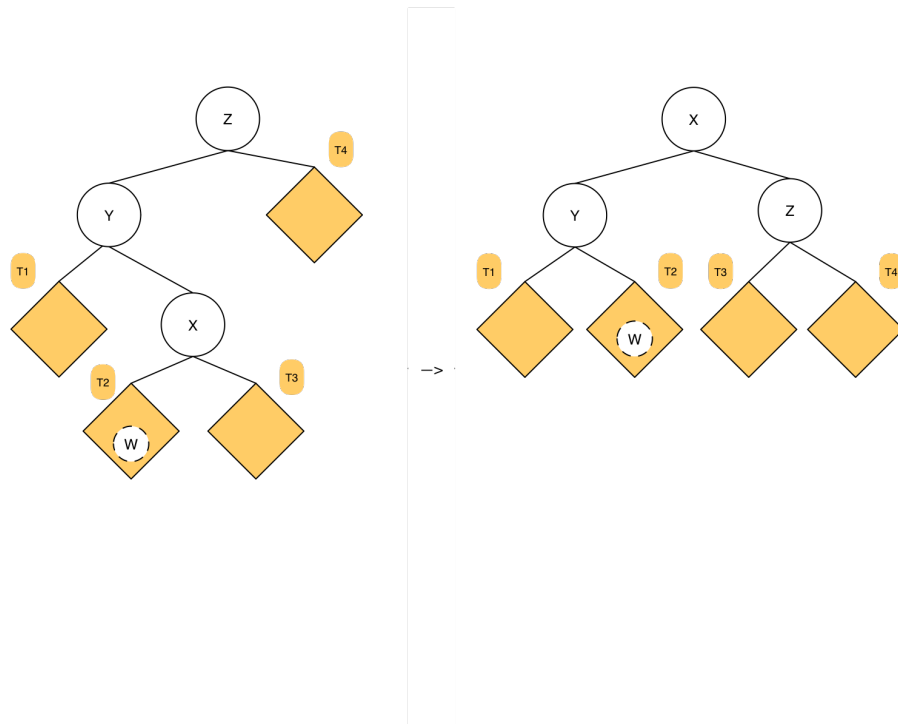


Figura 6.3: Doppia rotazione di X : prima attorno a Y poi attorno a Z

In questo caso abbiamo fatto una doppia rotazione; se avessimo avuto $X < Y < Z$ allora sarebbe bastata una singola rotazione di Y .

Siccome tutte le operazioni di ristrutturazione sono $\Theta(1)$, si può concludere che l'algoritmo di inserimento è $O(h)$.

Si può notare che l'algoritmo di inserimento riporta l'altezza del sottoalbero di radice Z all'altezza che aveva prima dell'inserimento del nodo W .

Passiamo alla **rimozione** `remove(k)`. Abbiamo visto i tre casi che si possono presentare, ci manca la parte di ristrutturazione dell'albero.

Individuiamo tre nodi:

- Z = il primo nodo sbilanciato risalendo da W (= il genitore del nodo rimosso) verso la radice (può coincidere con W);
- Y = il figlio di Z avente altezza massima (si dimostra che non è un antenato di W);
- X = il figlio di Y così definito:
 - quello di altezza massima, se i due figli di Y hanno altezze diverse;
 - altrimenti, X è il figlio di Y "che si trova dalla stessa parte di Y ", cioè X è il figlio destro di Y se e solo se Y è il figlio destro di Z ;
 - anche X non è un antenato di W .

Si dimostra che se Z esiste allora esistono anche X, Y e inoltre si dimostra che X non è una foglia.

A questo punto l'algoritmo effettua le stesse operazioni di ristrutturazione (rotazioni) viste nell'inserimento.

La differenza tra i due algoritmi è che (si dimostra), dopo una rimozione, il sottoalbero di radice Z è bilanciato ma non ha la stessa altezza che aveva prima della rimozione, può diminuire di un'unità (non si può fare meglio). Ciò significa che un antenato di Z potrebbe sbilanciarsi e quindi dobbiamo risalire l'albero e in caso dobbiamo ristrutturarlo.

Siccome il numero massimo di ristrutturazioni è $O(h)$, ciascuna delle quali è $\Theta(1)$, la rimozione ha prestazioni complessive $\Theta(h)$.

Conclusione: i metodi di rimozione e di inserimento sono $O(h)$ e non violano la proprietà di bilanciamento di un AVL, quindi possiamo concludere che realizzare una mappa ordinata mediante AVL permette di realizzare tutti i metodi con prestazioni $O(\log n)$.

Capitolo 7

Ordinamenti

Si dimostra che qualunque algoritmo di ordinamento che trae informazioni da confronti è, nel caso pessimo, $\Omega(n \log n)$. Non si può far meglio di così. In ogni caso possiamo confrontare questi algoritmi (detti ottimi) non asintoticamente ma puntualmente (analisi sperimentale) e scegliere quello con prestazioni migliori per il tipo di applicazione che vogliamo realizzare.

7.1 Bucket Sort

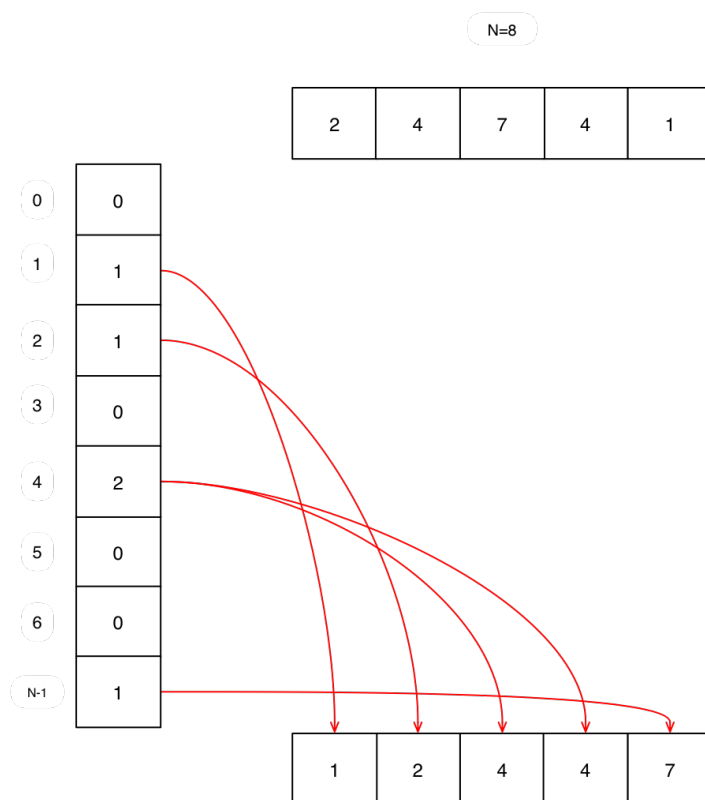


Figura 7.1: Bucket sort: array ausiliario usato come contatore

Il Bucket Sort è un esempio di algoritmo che non trae informazioni dai confronti. Viene utilizzato per ordinare liste di interi appartenenti a un intervallo del tipo $[0, N - 1]$.

Descrizione:

1. si parte da una lista (array o struttura concatenata) S di dimensione n ;
2. crea un array ausiliario A di N numeri interi inizializzati a 0 ($\Theta(n)$);
3. estraggo uno ad uno gli n valori da S : per ogni valore k incremento di una unità la cella k di A ($\Theta(n)$);
4. esamino le N celle di A in ordine di indice i crescente: inserisco in S (sequenza vuota) un numero $A[i]$ (> 0) di valori uguali a i ($\Theta(n + N)$);
5. la lista è ordinata (sul posto).

In questa descrizione abbiamo usato l'array ausiliario come contatore, se vogliamo riordinare una lista di coppie chiave valore (in cui sono presenti doppiioni), l'array ausiliario diventa un array contenente riferimenti di liste di coppie.

Notiamo che il passo 4 della descrizione richiede un tempo $\Theta(n + N)$, N è una costante che spesso non dipende da n , non la eliminiamo dalla notazione in quanto può essere molto influente. Se in un utilizzo pratico abbiamo $n \ll N$, otteniamo prestazioni temporali pessime e un'occupazione in memoria altrettanto pessima rispetto ad altri algoritmi di ordinamento.

In questi casi si potrebbe ridurre N : basta ricercare il valore *min* e *max*, porre l'indice in cui inserire il valore della lista da ordinare uguale a $N = \text{valore} - \text{min}$. In questo modo il valore *min* viene inserito nell'indice 0 e il *max* in $N = \text{max}$.

7.2 Paradigma divide and conquer

Il paradigma divide and conquer è un paradigma ricorsivo che permette di risolvere problemi complessi. Si basa sul seguente schema:

- **Caso base:** se la dimensione del problema è inferiore a una determinata soglia, risolvi il problema direttamente usando qualche strategia e termina.
- **Divisione:** suddividi i dati del problema in due o più insiemi disgiunti (cioè individua una partizione dei dati del problema).
- **Ricorsione:** risolvi ricorsivamente i sotto-problemi omogenei associati ai sottoinsiemi creati dal passo di divisione (ricorsione multipla, quasi sempre doppia).
- **Conquista:** usa le soluzioni dei sotto-problemi per generare (mediante una "fusione") la soluzione del problema originario.

Questo approccio viene utilizzato da diversi algoritmi (non solo di ordinamento), tra cui il Merge sort e il Quick Sort.

7.3 Merge-Sort

Viene inventato da John von Neumann nel 1945 (non erano ancora stati inventati i calcolatori). Il Merge-Sort permette di ordinare un array o una catena (non necessita dell'accesso casuale) usando il paradigma divide and conquer.

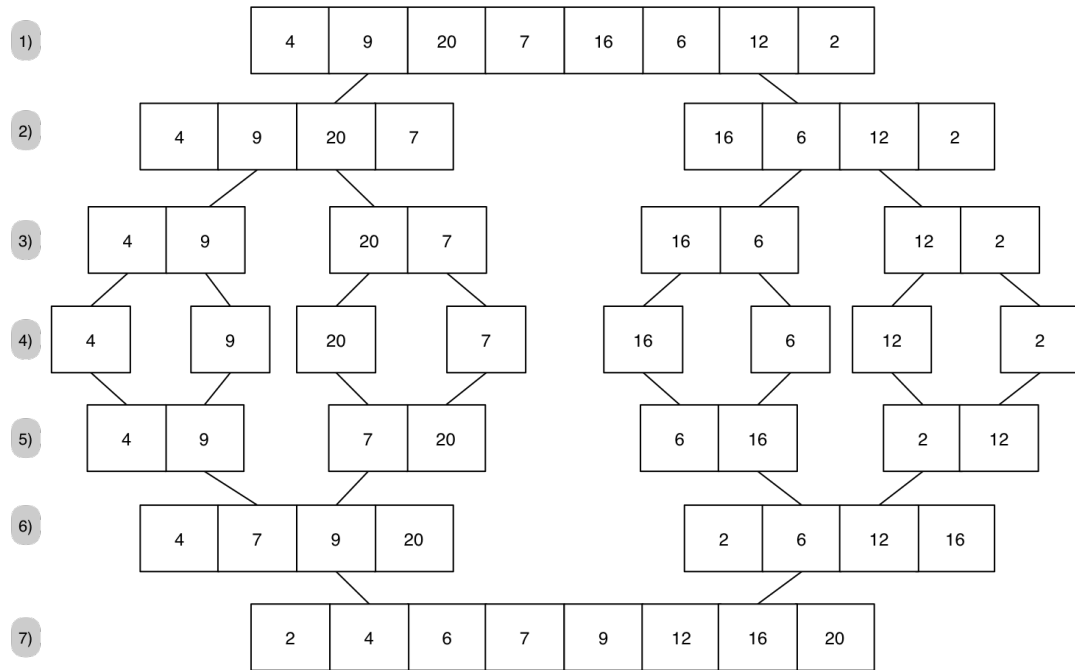


Figura 7.2: Merge sort

Descrizione:

si parte da una sequenza S di n oggetti appartenenti ad un insieme totalmente ordinato.

- Caso base: se S ha meno di 2 elementi, termina (la sequenza è ordinata).
- Divisione: estrai tutti gli elementi da S , inserendoli in due sequenze S_1 e S_2 :
 - S_1 contiene i primi $\lceil n/2 \rceil$ elementi di S ;
 - S_2 contiene i restanti elementi ($\lfloor n/2 \rfloor$ elementi).
- Ricorsione: ordina ricorsivamente (e separatamente) S_1 e S_2 .
- Conquista: fonda S_1 e S_2 in una sequenza ordinata, re-inserendo gli elementi in S (lista vuota).

L'ultimo passo (la fusione) richiede uno spazio aggiuntivo $\Theta(n)$ in quando viene generata una sequenza di dimensione n .

Prestazioni:

Per l'analisi delle prestazioni utilizziamo una strategia tipica degli algoritmi ricorsivi.

Dimostrazione. Indichiamo con $T(n)$ il tempo necessario a ordinare una sequenza di dimensione n con il merge-sort.

Per facilitare i calcoli, ipotizziamo che $n \geq 2$ sia una potenza di due: in questo modo $n/2$ è un numero intero, inoltre si ha $\lceil n/2 \rceil = \lfloor n/2 \rfloor = n/2$. (L'altro caso è analogo)

Per ordinare n elementi bisogna:

- costruire le due sottosequenze in un tempo $\Theta(n)$;
- invocare ricorsivamente merge-sort su due problemi di dimensione $n/2$ in un tempo $2T(n/2)$;

- effettuare le fusione in un tempo $\Theta(n)$

Otteniamo la seguente funzione in forma implicita

$$T(n) = \begin{cases} b & n < 2 \\ 2T(n/2) + cn & n \geq 2 \end{cases}$$

abbiamo ottenuto un'equazione ricorrente (matematica discreta). Nella formula, il termine cn , indica una funzione $\Theta(n)$ qualsiasi. Otteniamo:

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2[2T((n/2)/2) + c(n/2)] + cn \\ &= 4T(n/4) + 2c(n/2) + cn \\ &= 2^2T(n/2^2) + 2cn \end{aligned}$$

Applicandola k volte si ottiene: $T(n) = 2^kT(n/2^k) + kcn$. Sappiamo che quando $n/2^k = 1 \Leftrightarrow T(n/2^k) = b \Leftrightarrow k = \log_2 n$.

Sostituendo $k = \log_2 n$ in $T(n) = 2^kT(n/2^k) + kcn$ otteniamo:

$$\begin{aligned} T(n) &= 2^{\log_2 n}T(n/2^{\log_2 n}) + (\log_2 n)cn \\ &= nT(n/n)cn \log_2 n \\ &= nT(1) + cn \log_2 n \\ &= bn + cn \log_2 n \in \Theta(n \log n) \end{aligned}$$

■

L'algoritmo Merge-Sort è interessante anche in forma iterativa, ha prestazioni puntuali migliori rispetto a quello ricorsivo.

7.4 Quick-Sort

Algoritmo che viene scoperto da Charles A. R. Hoare nel 1961. Il Quick-Sort, come il Merge-Sort, si basa sul paradigma divide and conquer.

Descrizione:

Poniamo di dover ordinare una sequenza S di n oggetti.

- Caso base: se S ha meno di 2 elementi, termina (sequenza ordinata).
- Divisione: scegli un elemento $x \in S$ (chiamato *pivot*), estrai tutti gli elementi da S ed inseriscili in tre sequenze (tre partizini di S) secondo questo criterio:
 1. L (*less than*) contiene gli elementi minori di x ;
 2. E (*equal to*) contiene gli elementi uguali a x (solo x se sono tutti distinti);
 3. G (*grater than*) contiene gli elementi maggiori di x .
- Ricorsione: ordina ricorsivamente L e G .
- Conquista: re-inserisci gli elementi di S (sequenza vuota), prima quelli di L , poi quelli di E e infine quelli di G .

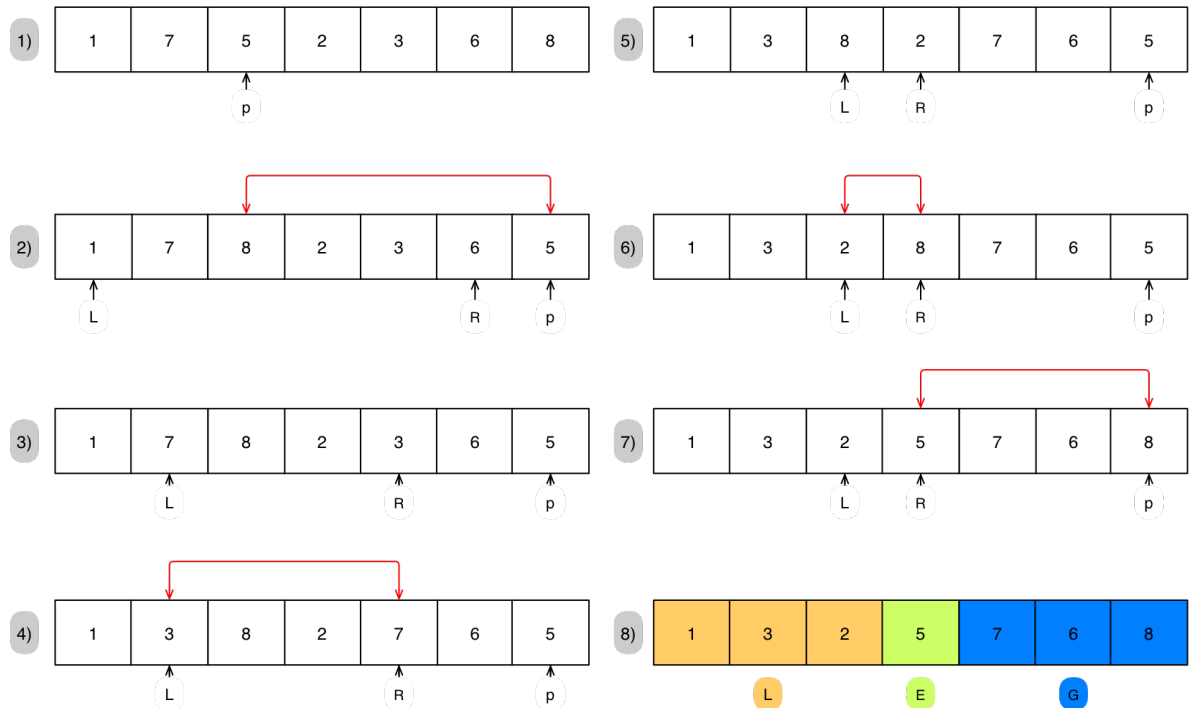


Figura 7.3: Quick sort sul posto

Se realizziamo un quick-sort sul posto, la fase di conquista rimane vuota, la lista è ordinata dopo l'ultima divisione.

Non abbiamo definito come viene scelto il pivot, in base ciò variano le prestazioni dell'algoritmo. Per ottenere prestazioni ottime, le liste L e G devono essere circa della stessa dimensione, quindi dobbiamo prendere un pivot che sia circa l'elemento mediano (l'elemento centrale se la lista fosse ordinata).

Si dimostra che se si preleva il pivot da una posizione prefissata nella sequenza, è sempre possibile progettare una sequenza che soddisfi il caso pessimo, in particolare il quick-sort diventa quadratico.

Randomize quick-sort:

Il quick-sort diventa interessante quando la scelta del pivot è casuale. In questo modo otteniamo prestazioni $O(n \log n)$ nel caso medio, con una costante moltiplicativa migliore di quella del merge-sort. Anche se nel caso pessimo rimane quadratico, si dimostra che le prestazioni sono $O(n \log n)$ con probabilità non inferiore a $1 - 1/n$. Quindi al crescere di n è sempre più probabile che il quick-sort non si trovi nel caso pessimo.

Quick-sort sul posto:

La fase di divisione può essere effettuata sul posto in un tempo $\Theta(n)$, le sottosequenze L , E e G si trovano all'interno della sequenza S stessa. Questa variante viene utilizzata solitamente con gli array (è molto più semplice da realizzare rispetto a una lista).

In questo modo lo spazio aggiuntivo richiesto è $\Theta(1)$ e non $\Theta(n)$ e la fase di conquista non deve fare niente. Le prestazioni asintotiche dell'algoritmo non migliorano, migliora solo la costante moltiplicativa.

```

1 // Algoritmo di divisione sul posto
2 // Per semplicità poniamo che gli elementi siano tutti distinti

```

```
3      Porto il pivot in ultima posizione (swap);
4      Individuo due indici:
5          - left: parte dal primo elemento di a[];
6          - right: parte dal penultimo elemento di a[];
7      while (left <= right)
8          while a[left] < pivot: left++;
9          while a[right] > pivot: right++;
10         if left < right // se non si sono incrociati
11             left e right si scambiano gli elementi (swap)
```

L'algoritmo quick-sort funziona anche se il pivot scelto non è un elemento dell'array. Per questo motivo l'algoritmo di divisione può essere utilizzato anche all'esterno dell'algoritmo di riordinamento.

Capitolo 8

Hasing

8.1 Tabelle

Se dobbiamo realizzare una mappa in cui le chiavi sono numeri interi compresi nell'intervallo $[0, N - 1]$ (N noto a priori), allora possiamo sfruttare una tabella. Concretamente si tratta di utilizzare una array in cui l'indice corrisponde alla chiave.

In questo modo, le operazioni di inserimento, rimozione e ricerca diventano $\Theta(1)$, l'unico svantaggio è che i metodi `keys()` e `value()` diventano $\Theta(N)$.

Un altro svantaggio riguarda la memoria: l'occupazione richiesta per contenere n coppie non dipende da n ma dipende linearmente dal valore della chiave massima, che in generale non ha alcuna relazione con n .

Per le tabelle, si definisce il **fattore di riempimento** $\lambda = n/N$, se $\lambda = 1$ significa che l'array non ha buchi.

Se N non è noto a priori si può realizzare una tabella ridimensionabile. In questo modo l'inserimento diventa $O(n)$ nel caso peggiore, questo perché non si raddoppia la dimensione dell'array quando la chiave da inserire è maggiore dell'indice massimo, ma si ridimensiona in modo che N sia il nuovo indice massimo. Rimangono i problemi dovuti agli sprechi di spazio.

8.2 Funzioni hash

Se la mappa che vogliamo realizzare non ha chiavi che sono numeri interi, dobbiamo trovare un modo per assegnare a queste chiavi un indice univoco. Questa trasformazione è di fatto una funzione che ha come dominio l'insieme delle chiavi della mappa, come codominio l'insieme degli indici validi per accedere a una tabella.

Queste funzioni prendono il nome di **funzioni hash**, il valore che producono viene chiamato chiave ridotta. Sfruttando queste funzioni hash, possiamo realizzare mediante tabella qualunque mappa.

Il compito di una funzione di hash viene svolto da due funzioni applicate una dopo l'altra:

- funzione di codifica;
- funzione di compressione.

8.2.1 Funzione di codifica

La funzione di codifica (di hash) ha come dominio l'insieme delle chiavi della mappa, come codominio l'insieme di numeri interi (in Java l'insieme dei valori rappresentati dal tipo di dato `int`).

In Java gli indici degli array sono rappresentati con il tipo `int` e devono essere non negativi, questo significa che c'è un indice massimo. Se l'insieme delle chiavi ha cardinalità maggiore di questo valore, allora chiavi diverse potrebbero avere lo stesso codice di hash. Questo fenomeno prende il nome di **collisione**.

Una funzione di hash che non genera collisioni si dice ottima.

Per risolvere le collisioni solitamente si sfrutta un array di liste (bucket).

Esempi di funzioni di codifica:

- **Funzioni di codifica ottime:** se le chiavi sono di tipo `byte`, `short`, `char` allora basta fare un cast implicito; se le chiavi sono di tipo `float` allora si sfrutta la rappresentazione in memoria (32 bit) come se fosse un `int` (32 bit). In Java usiamo il metodo `static int Float.floatToIntBits(float x)`. Queste funzioni non manipolano i bit, valori diversi hanno rappresentazioni binarie diverse, quindi otteniamo codici hash diversi.
- **Somma di componenti:** se le chiavi sono di tipo `double` o `long`, allora non esistono funzioni ottime. In questi casi usiamo una funzione hash che mescola le due porzioni da 32 bit:

```
1 static int hashCode(long i) { return (int)((i >> 32) + (int)i); }
```

- **Funzioni di codifica polinomiali:** se le chiavi sono stringhe non possiamo utilizzare la stessa funzione altrimenti gli anagrammi avrebbero tutti lo stesso codice (tante collisioni). In questi casi è meglio usare una funzione che tenga conto della posizione dei caratteri nella stringa, che dia quindi un peso alle posizioni. La stringa diventa un polinomio. Bisogna solo decidere quale base utilizzare. Sperimentalmente si osserva che il numero di collisioni si riduce drasticamente, se si utilizza una base che abbia pochi zeri nei suoi bit meno significativi.

8.2.2 Funzioni di compressione

La funzione di codifica permette di trasformare una chiave in un `int`, la funzione di compressione fa rientrare questo valore nel range $[0, N - 1]$.

Una funzione di compressione abbastanza efficace, prevede di assegnare al codice di hash h la seguente chiave ridotta: $q = |h| \bmod N$. Viene chiamata **compressione per divisione**. Sperimentalmente si dimostra che questa funzione di compressione è migliore se si sceglie un valore di N che sia un numero primo "abbastanza distante" da una potenza di due.

Anche le funzioni di compressione possono generare collisioni, le collisioni totali di una funzione hash sono date dalla somma delle collisioni delle due funzioni.

8.2.3 Gestione collisioni

Come abbiamo già anticipato, la migliore soluzione è quella di utilizzare un array di liste contenente le coppie chiave valore. Questa gestione delle collisioni si chiama **separate chaining**.

Prestazioni:

Le operazioni di inserimento, rimozione e ricerca devono: calcolare la chiave ridotta corrispondente alla chiave passata mediante la funzione di hash (codifica e compressione); accedere al bucket corrispondente alla chiave ridotta; ricercare/inserire/rimuovere nel bucket. Le prime due fasi sono $\Theta(1)$, le prestazioni dei metodi sono, nel caso pessimo, $O(n)$ (tutte le chiavi

hanno la stessa chiave ridotta).

Nel caso medio le prestazioni dei metodi dipendono linearmente dalla dimensione media di bucket: se la funzione di compressione è buona, le chiavi ridotte si distribuiscono uniformemente nell'intervallo $[0, N - 1]$ e quindi la dimensione media delle liste è $\lambda = n/N$.

Quindi le prestazioni nel caso medio dei metodi sono $O(\lambda)$, dove $\lambda = n/N$ è il fattore di riempimento.

Capitolo 9

Grafi

9.1 Introduzione

I grafi sono un tipo di dato astratto che rappresenta relazioni esistenti tra coppie di oggetti. Vengono indicati con $G(V, E)$, dove V è l'insieme di oggetti chiamati vertici (o nodi), mentre E è l'insieme di connessioni tra coppie di vertici, detti rami o archi.

Nei vertici memorizziamo dati, possiamo anche associare valori ai rami, in tal caso il grafo prende il nome di grafo pesato.

Nota che un ramo deve per forza collegare due vertici, mentre un vertice può non essere collegato a nessun ramo. In tal caso si parla di vertice isolato.

Carrellata di definizioni e termini che utilizzeremo:

- **ramo:** Per rappresentare un ramo $e \in E$, usiamo la coppia di vertici connessi dal ramo stesso, scriviamo $e = (u, v) \in E \Rightarrow u \in V, v \in V$;
- **ramo orientato:** quando l'ordine tra i vertici è importante, altrimenti il ramo è non orientato;
- **grafo orientato:** grafo in cui tutti i rami sono orientati;
- **grafo non orientato:** grafo in cui tutti i rami sono non orientati;
- **grafo misto:** grafo avente sia rami orientati che non;
- **vertici adiacenti:** due vertici collegati dallo stesso ramo. I due vertici sono gli **estremi** o **terminali** del ramo. Il ramo si dice **incidente** nei suoi due vertici;
- **grado di un vertice:** il grado di $v \in V \in G$ viene indicato da $deg(v)$ ed indica il numero di rami che sono incidenti in v . Se il vertice ha grado 0 si dice **isolato**. Tra i rami orientati incidenti in v , si distinguono: i **rami uscenti** da v , hanno come origine v e contribuiscono al conteggio di $outdeg(v)$; i **rami entranti** in v , hanno v come destinazione e contribuiscono al conteggio di $indeg(v)$.
In un grafo orientato $deg(v) = indeg(v) + outdeg(v)$;
- **grafo semplice:** grafo che non presenta rami paralleli (o multipli) e non presenta auto-anelli;
- **percorso o cammino:** sequenza di vertici e rami alternati modo che: la sequenza inizi con un vertice e termini con un vertice; ciascun ramo sia incidente nel vertice che lo precede e nel vertice che lo segue nella sequenza; nessun ramo compaia in due posizioni consecutive;
- **percorso semplice:** percorso in cui non passo più di una volta in uno stesso vertice;

- **percorso orientato:** percorso in cui i rami orientati vengono attraversati secondo la loro direzione;
- **ciclo:** percorso in cui vertice iniziale e finale coincidono. Esistono anche **cicli orientati**;
- **sottografo:** dato $G = \{V_G, E_G\}$, il grafo $H = \{V_H, E_H\}$ è un sottografo di G se $V_H \subseteq V_G \subseteq E_H \subseteq E_G$;
- **sottografo ricoprente:** H è un sottografo ricoprente di G se contiene gli stessi vertici di G (i rami non vengono considerati);
- **grafo connesso:** grafo in cui esiste un percorso tra due vertici distinti;
- **componenti connesse o isole:** sottografi di G connessi di dimensione massima;
- **grafo ad albero:** grafo connesso privo di cicli. Hanno il minimo numero di rami consentito. Struttura diversa dagli alberi radicati visti nei capitoli precedenti;
- **foresta:** grafo privo di cicli. Un albero è anche una foresta;
- **albero ricoprente:** sottografo ricoprente ad albero.

Alcune proprietà che utilizzeremo per analizzare le prestazioni degli algoritmi:

- $\sum_{v \in G} \deg(v) = 2m$ dove m è il numero di rami di G . Se G è orientato $\sum_{v \in G} \text{indeg}(v) = \sum_{v \in G} \text{outdeg}(v) = m$;
 - Se G è un grafo semplice:
 - se non è orientato $m \leq n(n-1)/2$, se $=$ si dice completo;
 - se è orientato $m \leq n(n-1)$;
- $\Rightarrow m \in O(n^2)$, cioè il numero dei rami di un grafo ha un limite superiore.
- se G semplice, non orientato:
 - se è connesso allora $m \geq n-1$;
 - se è una foresta allora $m \leq n-1$;
 - se è un albero allora $m = n-1$;

nota che non sono $\Leftrightarrow$ ma $\Rightarrow$.

9.2 L'interfaccia Graph

```

1  public interface Edge<T> extends Position<T> { /*etichette*/ }
2  public interface Vertex<T> extends Position<T> { /*etichette*/ }
3  public interface Graph<V,E> { // non orientato
4      // ispezione
5      int numEdges();
6      int numVertices();
7      Iterable<Edge<E>> edges();
8      Iterable<Vertex<V>> vertices();
9      Iterable<Edge<E>> incidentEdges(Vertex<V> v);
10     Vertex<V> opposite(Vertex<V> v, Edge<E> e);
11     Vertex<V>[] endVertices(Edge<E> e)
12     boolean areAdjacent(Vertex<V> u, Vertex<V> v)

```

```

13      // modifica
14      V replace(Vertex<V> v, V o); // il dato
15      E replace(Edge<E> e, E o); // il dato
16      Vertex<V> insertVertex(V o); // inserito isolato
17      V removeVertex(Vertex<V> v); // di solito rimuove
18      // anche i rami incidenti, altrimenti
19      // lancia eccezione se ha rami incidenti
20      Edge<E> insertEdge(Vertex<V> u, Vertex<V> v, E o);
21      E removeEdge(Edge<E> e);
22  }

```

9.3 Attraversamenti di grafi

Un attraversamento di un grafo è una procedura che permette di visitare tutti i suoi n vertici e tutti i suoi m rami, una e una sola volta.

L'attraversamento più semplice che possiamo realizzare è quello che scandisce le due liste fornite dai metodi `vertex()` e `edges()`. Questo è utile se non ci interessano informazioni sulla topologica del grafo, in caso contrario dobbiamo sfruttare un attraversamento topologico.

9.3.1 Attraversamento DFS

Attraversamento in profondità, per motivi storici viene chiamato *depth-first-search* da cui DFS. In sintesi, parte da un vertice qualunque del grafo e ceca di andare "il più lontano possibile" per poi "tornare sui suoi passi" (backtraking) e percorrere altre strade.

```

1      // DFS ricorsivo
2      DFS(G, s)
3          s.setLabel(VISITED) // visito il vertex
4          for each e ∈ G.incidentEdges(s) // per ogni ramo incidente
5              if e.getLabel() == UNEXPLORED // non esplorato
6                  e.setLabel(VISITED) // visito il ramo
7                  w ← G.opposite(s, e) // prelevo il vertex opposto a s
8                  if w.getLabel() == UNEXPLORED // se non e' stato esplorato
9                      DFS(G, w) // ricorsione su w
10                 // else: sono in un vicolo cieco => torno al chiamante

```

Si dimostra (per assurdo) che l'algoritmo effettua un attraversamento del componente connesso C a cui appartiene il vertice di partenza s . Quindi se il grafo è connesso, l'algoritmo attraversa ogni suo nodo e ogni suo ramo.

Possiamo utilizzare questo algoritmo per verificare diverse proprietà topologiche di un grafo. Per esempio:

```

1      // grafo connesso?
2      // main
3      // setto i rami e vertici a UNEXPLORED
4      for each u ∈ G.vertices()
5          u.setLabel(UNEXPLORED)
6      for each e ∈ G.edges()
7          e.setLabel(UNEXPLORED)
8      connectedComponents = 0
9      for each v ∈ G.vertices()
10         if (v.getLabel() == UNEXPLORED)
11             connectedComponents++;

```

```
12 DFS(G, v) // invoco DFS ricorsivo
```

Se `connectedComponents == 1` $\Rightarrow$ il grafo G è connesso.

```
1 // ricerca di spanning tree: modifico DFS ricorsivo
2 // lancio l'algoritmo con il main sopra
3 // DFS ricorsivo
4 DFS(G, s) // ricorsivo
5   s.setLabel(VISITED)
6   for each e ∈ G.incidentEdges(s)
7     if e.getLabel() == UNEXPLORED
8       w ← G.opposite(s, e)
9       if w.getLabel() == UNEXPLORED
10        e.setLabel(DISCOVERY)
11        DFS(G, w)
12     else // sono in un vicolo cieco
13       e.setLabel(BACK)
```

Tutti i rami che hanno etichetta DISCOVERY individuano una spanning forest di G . Se G è connesso, individuano uno spanning tree.

Con lo stesso algoritmo possiamo verificare se G è ciclico, se infatti esiste almeno un ramo con etichetta BACK, implica che G è ciclico.

Prestazioni: Poniamo che tutti i metodi siano ottimi. Abbiamo:

- l'inizializzazione dei rami e vertici a UNEXPLORED è $\Theta(n + m)$;
- nella parte ricorsiva:
 - `incidentEdges()` viene invocato una e una sola volta per ogni vertice. Se `incidentEdges(v) =  $\Theta(deg(v))$` allora il contributo di tutte le invocazioni è $\Theta(m)$ (la somma di tutti i gradi di tutti i vertici);
 - `opposite()` viene invocato una e una sola volta per ogni ramo del grafo, quindi il contributo totale è $\Theta(m)$.

Quindi, se tutti i metodi sono ottimi, il DFS è $\Theta(n + m)$ cioè è ottimo in quanto un attraversamento non può avere prestazioni migliori.

I metodi di accesso e modifica delle etichette sono ininfluenti, infatti danno un contributo $\Theta(n + m)$.

9.3.2 Attraversamento BFS

L'attraversamento BFS (*breadth-first-search*) lavora in ampiezza a differenza del DFS che lavora in profondità. Il BFS visita prima i vertici adiacenti al vertice iniziale, poi si allontana "a macchia d'olio", per cerchi concentrici. Durante il funzionamento, oltre a decorare vertici e rami con le etichette, assegna un livello a ciascun vertice (non c'è bisogno di memorizzarla nei vertici), questo comportamento può essere molto utile per risolvere alcuni problemi.

L'attraversamento BFS permette di verificare le stesse proprietà dei grafi che abbiamo visto negli algoritmi che sfruttano il DFS. La sostanziale differenza tra i due attraversamenti consiste nell'ordine in cui vengono visitati i vertici.

```
1 BFS(G, s) // iterativo
2   L0 ← new empty sequence
3   L0.addLast(s)
4   s.setLabel(VISITED)
```

```

5      i ← 0
6      while !Li.isEmpty()
7          Li+1 ← new empty sequence // lista temporanea
8          for each v ∈ Li
9              for each e ∈ G.incidentEdges(v)
10                 if e.getLabel() == UNEXPLORED
11                     w ← G.opposite(v, e)
12                     if w.getLabel() == UNEXPLORED
13                         e.setLabel(DISCOVERY)
14                         w.setLabel(VISITED)
15                         Li+1.addLast(w) // aggiungo alla lista temporanea
16                     else
17                         e.setLabel(CROSS)
18
19      i ← i + 1

```

Si dimostra che $\text{BFS}(G, s)$ attraversa il componente connesso a cui appartiene il vertice iniziale s $\text{BFS}(G)$ (invocazione per ogni vertice di G) attraversa il grafo. Se G è rappresentato mediante adjacency list, l'algoritmo ha prestazioni ottime, cioè $\Theta(n + m)$.

Si dimostra che, se il vertice v viene posto al livello k dal BFS iniziato in u , allora il percorso minimo tra u e v ha lunghezza k . In altre parole, il BFS è in grado di trovare un percorso minimo di rami DISCOVERY che collegano due vertici u e v .

Si dimostra che i rami DISCOVERY di BFS costituiscono una spanning forest di G (come per DFS), ciascuno spanning tree viene detto albero BFS.

9.4 Implementazione di grafi

Esistono diverse implementazioni dell'interfaccia `Graph`, sono tutte variazioni sul tema delle tre seguenti: Edge list, Adjacency list, Adjacency matrix. Tutte e tre hanno `numEdges()`, `numVertices()`, `opposite()`, `endVertices()`, `replace()`, `insertEdge()` e `removeEdge()` che sono $\Theta(1)$, quindi ottimi.

- **Edge list:** usa una lista di vertici V e una lista di rami E . Ogni vertice è un oggetto che contiene un dato e un riferimento alla propria posizione nella lista V (*location-aware*). Ogni ramo è un oggetto che contiene un dato, un riferimento alla propria posizione nella lista E (*location-aware*) e riferimenti ai propri vertici terminali. Si tratta di un'implementazione poco efficiente in quanto non è agevole l'accesso ai rami partendo da un vertice (il contrario è efficiente).
- **Adjacency list:** simile all'implementazione di Edge list. Ogni vertice punta a una propria "lista di incidenze", che contiene riferimenti ai rami incidenti del nodo. Ogni ramo punta anche alla propria posizione nelle liste di incidenze relative ai suoi due vertici terminali (*location-aware* in entrambe le liste). Si tratta dell'implementazione più efficiente.
- **Adjacency matrix:** utile per quei problemi che traggono vantaggio dalle prestazioni del metodo `areAdjacent()` in quanto lo rende costante. Arricchisce la edge list con una matrice A che rappresenta, in ogni casella, coppie di vertici adiacenti: ad ogni vertice viene aggiunto un indice intero tra 0 e $n - 1$, usato come indice di riga e di colonna nella matrice A ; ogni cella di A contiene un riferimento al ramo che collega i due vertici corrispondenti ai suoi indici, se sono adiacenti, altrimenti contiene `null`.

9.5 L'algoritmo di Dijkstra

Prima introdurre l'algoritmo, introduciamo alcune definizioni:

- **lunghezza di un percorso:** somma dei pesi dei rami;
- **distanza tra due vertici:** lunghezza minima di un percorso che collega i due vertici (appartenenti alla stessa componente connessa). Se i due vertici non appartengono alla stessa componente connessa, si dice (per convenzione) che la loro distanza è ∞ .

L'algoritmo di Dijkstra permette di trovare il percorso minimo tra un vertice assegnato s e un qualsiasi altro vertice del grafo pesato. L'algoritmo può essere diviso in due fasi:

1. assegnare ad ogni vertice del grafo l'etichetta che indica la distanza di quel vertice dal vertice di partenza s (questo è l'algoritmo di Dijkstra);
2. costruire l'albero dei percorsi minimi da s , in questo modo possiamo individuare il percorso minimo tra il vertice s e qualsiasi vertice del grafo.

L'idea:

- si fa crescere attorno ad s una nuvola C di vertici che, passo dopo passo, includerà tutti i vertici del grafo;
- C inizialmente contiene solo s in quanto è il vertice che ha distanza minima $d(s) = 0$ da s . Ogni volta che un vertice entra in C , viene etichettato con la sua distanza da s ;
- ad ogni iterazione viene inserito in C il vertice u che ha $d(u)$ minima, la sua etichetta viene aggiornata (e mai più toccata);
- dopo aver inserito u bisogna aggiornare le etichette dei nodi adiacenti a u . Se $u \in C$ e $v \notin C$ sono adiacenti e w è il peso del ramo $e = (u, v)$, allora:
 - se $d(u) + w < d(v)$ significa che $d(u) + w$ è una migliore approssimazione della distanza di v da s quindi poniamo $d(v) = d(u) + w$. Questa procedura mediante approssimazioni successive si chiama **rilassamento**. L'aggiornamento mediante rilassamento è monotono, può soltanto far diminuire la distanza stimata;
- i vertici esterni a C vengono mantenuti in una `AdaptableMinPQ`.

Per far sì che l'algoritmo, oltre a calcolare la distanza di s di tutti i suoi vertici, trovi anche l'albero dei percorsi minimi avente radice s , dobbiamo:

- introdurre l'etichetta `parent` che, alla fine dell'esecuzione dell'algoritmo, in ogni vertice indica il ramo che lo collega al suo genitore nell'albero dei percorsi minimi;
- il vertice s ha etichetta `NULL` in quanto è la radice;
- ad ogni rilassamento bisogna aggiornare le etichette "genitoriali": se il rilassamento coinvolge $u \in C$ e $v \notin C$, si individua un nuovo percorso minimo tra s e v che passa per u , quindi u è il nuovo genitore (non ancora definitivo) di v nell'albero dei percorsi minimi. Quindi `v.setParent(u)`, e è il ramo che rende v adiacente a u ;
- quando un vertice entra nella nuvola $\Rightarrow$ non subisce più rilassamenti $\Rightarrow$ la sua etichetta "genitore" diventa definitiva.

```
1  Dijkstra(G, s)
2      q = HeapAdaptableMinPQ()
3      s.setLocationAware(q.insert(0, s))
4      s.setLabel(0)
5      s.setParent(NULL) // per i percorsi, non "Dijkstra puro"
6      for v ∈ G.vertices()
7          if v != s
8              v.setLocationAware(q.insert(+∞, v))
9              v.setLabel(+∞)
10     while !q.isEmpty()
11         entry = q.removeMin()
12         label = entry.getKey()
13         vertex = entry.getValue()
14         if label == +∞
15             break // G non e' connesso
16         for edge ∈ G.incidentEdges(vertex)
17             z = G.opposite(vertex, edge)
18             newLabel = label + edge.getLabel()
19             if newLabel < z.getLabel() // rilassamento
20                 z.setLabel(newLabel)
21                 z.setParent(edge) // per percorsi, non "Dijkstra puro"
22                 q.replaceKey(z.getLocationAware(), newLabel)
```